**University of Stuttgart**
Institute of Industrial Automation and Software Engineering

# Conception and prototypical implementation for mapping the external system's context to its dynamic behavior

Research Project fa-3400

Submitted at the University of Stuttgart by

**Ahmed Mahfouz**

INFOTECH

Examiner: Prof. Dr.-Ing. Dr. h.c Michael Weyrich

Supervisor: Nada Sahlab, M.Sc.

30.12.2022

# Table of Contents

# Table of Figure

# Table of Tables

# Table of Abbreviations

| | |
|---|---|
| IoT | **I**nternet **o**f **T**hings |
| CPS | **C**yber-**P**hysical **S**ystem |
| iCPS | **I**ntelligent **C**yber-**P**hysical System |
| CP-AS | **C**yber-**P**hysical **A**utomation System |
| AI | **A**rtificial **I**ntelligence |
| FBS | **F**unction-**B**ehaviour-**S**tructure |
| FOL | **F**irst **O**rder **L**ogic |
| RDF | **R**esource **D**escription **F**ramework |
| LPG | **L**abeled **P**roperty **G**raph |
| LED | **L**ight **E**mitting **D**iode |
| PCB | **P**rinted **C**ircuit **B**oard |
| LCD | **L**iquid **C**rystal **D**isplay |
| PIR | **P**assive **I**nfrared **S**ensor |
| XDK | **C**ross **D**omain **D**evelopment **K**it |
| JSON | **J**ava**S**cript **O**bject **N**otation |
| CSV | **C**omma-**S**eparated **V**alues |
| UDP | **U**ser **D**atagram **P**rotocol |
| MQTT | **MQ** **T**elemetry **T**ransport |
| PWM | **P**ulse **W**idth **M**odulation |
| GPIO | **G**eneral-**P**urpose **I**nput/**O**utput |
| CPU | **C**entral **C**ontrol **U**nit |

# Glossary

| | |
|---|---|
| **Middleware** | Software by which software platforms and devices are connected to unify the service provided to the user. |
| **Context-Aware System** | Automation systems with the ability to relate their context at runtime to the system's operation. So, context parameters can be acquired, and included in the decision-making process. |
| **CPS** | Systems in which software and hardware components are integrated to perform a certain task. The tasks are highly automated and can be distributed between more than one agent. |
| **IoT Platform** | A platform by which physical entities with network connectivity are connected such as sensory networks, gateways, and end-user applications. |
| **Heterogeneous Data** | Data with various resources and data types. It may have some ambiguity, missing, or redundant values. |
| **Cypher** | One of the query languages is based on patterns. It is also designed to work with different versions of these patterns. |
| **Neo4j** | Data Platform for Graph Database applications by which data can be stored and managed. |

# Abstract

Automation systems are systems in which sensors, controllers, and actuators are integrated to automate a technical process without or with the least human interference. This system objective requires some features an automation system should have. The automation systems being exposed to environmental dynamic changes leads to the idea of developing intelligent and adaptive automation systems. During runtime, context information which defines all information characterizing the system components within the application interaction can be collected to increase the system’s ability to adapt.

Context-awareness can be defined as the system's being able to recognize its operational conditions and map them to the system's operation. Context-aware systems are considered here as automation systems with the ability to relate their context at runtime to the system's operation. So, context parameters can be acquired, and included in the decision-making process. Context-awareness can optionally enable both adaptation and system or fault analysis improvement.

One of the most important aspects of achieving system adaptability is to model and use the external context to which the system is exposed at runtime and map this context information to the system’s dynamic behavior. This can help understand and relate the system’s ambiguous behavior and achieve an optimized system operation.

In this research project, a concept for mapping the system's external context to its dynamic behavior is developed in a form of a mathematical equation, based on the data stored and queried in a graph data model. Then a control rule is designed to achieve the system's adaptability to its external context. Finally, a use case is implemented as a proof of concept.

**Key Words:** *context-awareness, context modeling, graph modeling, external context mapping, control*

# 1 Introduction

## 1.1 State of the Art

In 2006, the United States National Science Foundation introduced the concept of cyber-physical systems [1]. However, the history of these systems' appearance is much longer and can be related to the start of cybernetics, which was defined as the science of control and communication between machines and humans [1].

Different elements of scientific theories and engineering disciplines such as cybernetics, distributed control, embedded systems, sensor networks, control theory, and systems engineering are combined by cyber-physical systems (CPS) [1]. Digital and physical components are optimally integrated by these theories and disciplines to allow the maximum benefits of performance, cost, and life-cycle sustainability [1].

One possible definition of cyber-physical systems is the systems in which software and hardware components are integrated to perform a certain task. The tasks are highly automated and can be distributed between more than one agent [1]. Recently, research trends introduced new functionalities of artificial intelligence (AI) and machine learning that help in building reliable systems, which are called 'intelligent' cyber-physical systems (iCPS) [1].

Cs-Communication (Clear, Coherent, and Complete Communication), control, and computing are the three fundamental attributes required to have a system in which computations are affected by physical attributes and vice versa [1]. Sensor networks and embedded computing are integrated into a cyber-physical system, so the physical environment is monitored and controlled with feedback loops allowing external events to activate either the system computing, control, or communication [1].

This study aims to model the internal and external context affecting a context-aware automated pill dispenser system. It also aims to map the system's external context to its internal dynamic behavior at runtime in a form of a mathematical equation or a state machine that models the system's output discrepancy. Finally, a control rule is developed to control the system's output, so its discrepancy is minimized and the system adaptation to the change in its dynamic behavior is achieved. One of the challenges of achieving these goals is how to store and manage the system's context data, so they are modeled and can be mapped to the system's output discrepancy. Another challenge is how to automate the system's response to its current affecting context. This can lead to the project objective which can be the system's adaptation achievement.

Chapter 1 will introduce the project's different phases and their related tasks. Chapter 2 will introduce basic definitions of context-aware systems, context modeling, and graph modeling. Chapter 3 will introduce the developed concept of mapping the

system's external context to its dynamic behavior and the control loop of achieving system adaptation via introducing different use cases. Chapter 4 will introduce a prototype implementation for the alarm function for the pill dispenser. Chapter 5 will introduce the prototype integration into the system's current implementation, Chapter 6 will conclude the work and recommend future work.

## 1.2 Project Definition

This project mainly consists of four phases:

1. Definition Phase

The definition phase includes three main tasks:

- Defining the system requirements discussed with the supervisor and summarizing them in the System Requirements Specification document.
- Understanding the basic concepts of context-aware systems, context modeling, and graph models and summarizing them in the basics document.
- Analysis and updating of the existing model.

2. Conception Phase

The conception phase includes two main tasks:

- Designing a concept for modeling the relationship between the system's external context (user and environment) and its internal context (functions, behavior, and structure) based on the FBS framework of Gero.
- Designing a software architecture for applying the designed concept on the system graph data model in the form of cypher queries.

3. Prototype Phase

The prototype implementation phase includes two main tasks:

- Implementing the software components for the developed concept.
- Implementing the software component for the use case used as a proof of concept.

4. Integration and Test phase

The integration phase includes two main tasks:

- Integration of the implemented system after testing the system middleware.
- Deriving a user manual by which the way to test and run the system is explained.

# 2 Basics

In this section, the fundamental concepts of context-aware CP-AS, context modeling, labeled property graphs, FBS framework, and cypher graph query language, will be summarized. Then some basic decisions relating to these aspects will be taken to be the basis of the concept that will be developed in this project.

## 2.1 Cyber-Physical Automation Systems (CP-AS)

CP-AS are dynamic and collaborative systems that consist of actuators and sensors connected to the computational entities which can learn via information acquiring and processing to conclude an action. The CP-AS physical part includes the controlled system, sensors, and actuators, while the computational resources and services which enable control, analysis, decision-making, storage, and monitoring represent the cyber world of the CP-AS [2]. These computational resources make the CP-AS able to learn, interpret and process context into meaningful knowledge which leads to intelligence enhancement [3].

The following sub-section will introduce context-aware cyber-physical automation systems as a type of cyber-physical automation system that can adapt to the dynamic change of their environment.

### 2.1.1 Context for Cyber-Physical Automation Systems

Context defines all information characterizing the system situations within the application interaction [4]. This information is selected intentionally based on the application and the provided service, so the system is adaptive to user interaction [5]. In the autonomous driving domain, the context was defined as the relevant information about the vehicle's current environment relative to the vehicle control unit [6]. Similarly, the industrial definition for context can be the information on the interaction between the system component and its control unit. This information can be for example material objects and immaterial states [7]. From this definition, one can conclude that the context is observable from distributed sources [5]. Context changes due to time and space restrictions [5]. Context is unknown as it is not directly measured [5]. Context is relative depending on one's point of view [5].

Context has a lifecycle of data gathering and processing. It is also classified based on its source, relevance, or type which leads us to a new definition for the CPS context as a collection of time-continuous data about CPS representing networked components sharing a constrained target or circumstance to help understand the CPS in a better way. When the CPS context is modeled it must be system-centering. The complex structure and components relationship should be considered. Ontological modeling allows a formal description of the context, but a complex hierarchical structure. Fuzzy logic can add probabilistic modeling. Graph-based models can show the component's

similarities and event relationships and interactions. They are also easier to access and managed [5].

### 2.1.2 Context-Aware Cyber-Physical Automation Systems

CP-AS can be described as a context-aware system. Due to being exposed to dynamic operating conditions such as dynamic environment, networking, and other systems or user interactions, the CP-AS enhances its decision-making capabilities during runtime to adapt to these conditions. Context-awareness can be defined as the system's being able to recognize its operational conditions and map them to the system's operation. Recent studies indicated that context-aware functionalities add more value to the system features. However, the lack of experience and standards makes it more challenging to design context-aware systems. The acquisition and processing of heterogenous data have been improved due to the evolution of IoT, CP-AS, and AI, which increased context-awareness and enhanced the decision-making and performance of the overall system [8].

Context-aware systems are considered here as automation systems with the ability to relate their context at runtime to the system's operation. So, context parameters can be acquired, and included in the decision-making process. Context-awareness can optionally enable both adaptation and system or fault analysis improvement [8].

### 2.1.3 Requirements for Context-Aware Systems (CP-AS)

Due to the heterogeneity of the context as being dynamic and acquired from different data sources, a context-aware system should be able to acquire heterogeneous context data. For an efficient application of context, it should also be able to extract a unified model which represents the collective data. For knowledge conclusion, reasoning and analyzing the context model are also required. Generally, context acquisition, context modeling, and reasoning component are the three components of context-aware systems. A design approach for context-aware CP-AS was proposed based on these requirements. The heterogeneity of the data acquisition was considered via an IoT integration platform. The context was modeled using semantic labeling and graph-based approaches which will be introduced in sections 2.2.3 and 2.2.4, so the dynamic relations between context entities are highlighted. A graph database managed and stored the context graph. Standardized and scalable access to the managed context model was enabled via a middleware design approach considering relevant contextual elements, to adapt and control the system [5].

## 2.2 Context Modeling for Cyber-Physical Systems

Improving methods for dynamic modeling and learning from context is necessary to capture the increased system's dynamism. Modeling approaches will be discussed in the subsequent sections.

### 2.2.1 Context Modeling Approaches

This section provides a short overview of some of the most popular approaches to context modeling.

#### 2.2.1.1 Key-Value Pair Model

Key-value pairs make up a straightforward tuple of data. The context information is associated with a particular key to facilitate quick lookup by using a matching algorithm. Although these pairs are simple to handle, they lack structuring, which prevents effective context retrieval [9].

#### 2.2.1.2 Markup Scheme Model

A hierarchical data structure of markup tags, attributes, and content are included in markup scheme models. For example, Context Meta Language (ContextML) is a markup-based system that represents context metadata as well as context information [9].

#### 2.2.1.3 Graphical Model

This model enables graphical diagrams that can specify relationships. Unified Modelling Language (UML), Entity Relationship Model (ERM), and Object Role Model are three examples of frequently used models (ORM). While ERM and ORM are used for conceptually designing and querying databases, the UML is a standardized general-purpose language that expresses various types of context information in its graphical notation. The interoperability of various storage databases used in the low-level graphical model itself, however, presents a problem [10].

#### 2.2.1.4 Object-Oriented Model

Class hierarchies and relationships are used by the object-oriented model to represent context data, and encapsulation, inheritance, and reusability are incorporated into context expression. By using inheritance mechanisms, context access can be granted to instances. An entity, which is the fundamental element, serves as the topic of structured context data. Attributes, also known as associations, are how an entity is connected to other entities. This method puts a lot of pressure on developers to understand the entire context taxonomy [10].

#### 2.2.1.5 Logical-Based Model

Context is referred to as facts, expressions, and rules in a logic-based model. The data in this model can be updated, added, or removed as needed. Thus, this model provides a high level of formality. This model has been adopted by numerous applications. This model is a helpful tool for determining context consistency as well as supporting the reasoning task [10].

#### 2.2.1.6 Ontology-Based Modeling

Ontological modeling is a term for a conceptual, but abstract, view of reality. Object-oriented techniques could also be used to describe the relationships within. However, in the context of the semantic web, an ontology is typically described using languages standardized by the W3C. The Web Ontology Language (OWL) and the Resource Description Framework Schema (RDF-S) are the two most applicable standards ([9], [11], [12]).

The next section will introduce the FBS design framework as recommended approach for mapping the system's external context to its dynamic behavior during runtime by driving the expected system's behavior from its functions and comparing it to its actual behavior derived from the system's structure.

### 2.2.2 Function-Behavior- Structure framework

Most agent-based systems rely on classical models and theories by which the system environment is assumed to be fixed, and well-defined. For this reason, models do not include prior knowledge and support changing environments. Function-behavior-structure (FBS) framework by Gero shown in Figure 1 is one of these models. The model adapts the agent's view of the environment and changes based on how the agent interacts with the environment [14].

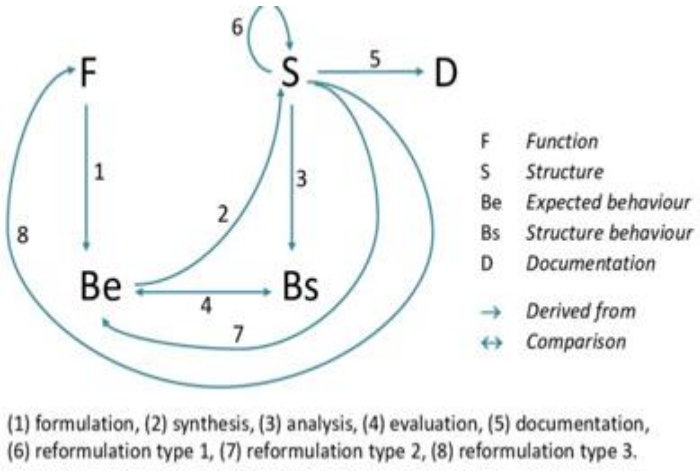


**Figure 1: FBS design paradigm by Gero [13]**

#### 2.2.2.1 The FBS framework

The FBS framework consists of three types of variables for different aspects of the design:

- Function (F) variables: Defines the object's functionality.
- Behavior (B) variables: Defines the attributes derived or expected from the structure (S) variables.
- Structure (S) variables: Defined the object's components and their internal relationships.

Connections between these three aspects are designed by an expert designer, where functions are mapped to behavior and behavior derived from structure. The function is not connected directly to the structure. Function, behavior, and structure are connected via a set of 8 processes as shown in Figure 1, described as follows:

1. Formulation: Maps the design requirements represented in function (F) into expected behavior (Be).
2. Synthesis: Maps the expected behavior (Be) into a solution structure (S).
3. Analysis: Maps the synthesized structure (S) to the actual behavior (Bs).
4. Evaluation: Compute the difference between the behavior derived from structure (Bs) with the expected behavior and decides if the design solution is accepted.
5. Documentation: Resulting in the design description (D) for building the product.
6. Reformulation type 1: Tracks the changes in the design state space from the structure variables' point of view.
7. Reformulation type 2: Track the changes in the design state space from the behavior variables' point of view.
8. Reformulation type 3: Track the changes in the design state space from the function variables' point of view.

#### 2.2.2.2 FBS-based Ontology Modeling

In the FBS model, the designed ontology is formalized as a function model and behavior model. It is equivalent to knowledge that is descriptive, procedural, manual, design guide, and individual in nature. In the designed ontology, first, a function model is created. Then, the functions that we want to implement drive the design behaviors. A behavior model includes actual design processes like design behaviors, practices, formulas, parameters, inputs, and outputs, among others [14].

1. Domain Ontology Modeling
   In the FBS model, the domain ontology corresponds to the structure model. In terms of knowledge classification, it also relates to descriptive knowledge and individual knowledge. It includes specific descriptive information like design domain classifications, component makeups, concrete people, material, definitions, benefits and drawbacks, principles, functions, experiences, limitations, etc.

2. Design Ontology Modeling
   The primary design task, which is broken down into several smaller tasks, is where we begin. They are made up of knowledge from design guides, including design notices, principles, indexes, requirements, etc. Knowledge of the design guide informs design processes. Each process includes data on input/output, design techniques, formulas, design diagrams, related individuals, tools, etc.

In the concept phase, this FBS framework will be included in adjusting the system’s current context model by adding labels to its nodes as well as new nodes.

### 2.2.3 Semantic Technologies

Semantic technologies cover a broad range of techniques and tools for representing, managing, exchanging, and processing semantic data. The World Wide Web Consortium (W3C) has developed several related technology standards as part of its Semantic Web and Web of Data initiatives. They consist of infrastructure and methodology components like metadata and infrastructure, as well as languages like RDF, OWL, and SPARQL [15].

Several practical issues are the focus of semantic technologies research. The fields of knowledge representation and reasoning are closely related to the specification and processing of ontologies. OWL and RDF have numerous significant applications that are either unrelated to or completely unrelated to the web [15].

### 2.2.4 Graph Data Modeling

#### 2.2.4.1 Knowledge Graphs

Initially, we have a CP-AS which is the pill dispenser represented in a context model called a knowledge graph. Knowledge graphs are interconnected sets of facts that present real-world objects, events, or relationships in a way that is both machines- and human-understandable. The organizing principle is used by knowledge graphs to enable users to relate to the underlying data. To support reasoning and knowledge discovery, the organizing principle provides us with a second layer of organizing data (metadata) that adds connected context [16].

#### 2.2.4.2 Labeled Property Graph (LPG)

A labeled property graph model is much like an object model or an entity-relationship diagram. A user can describe any domain as a connected graph of nodes and relationships with properties and labels using graph data modeling.

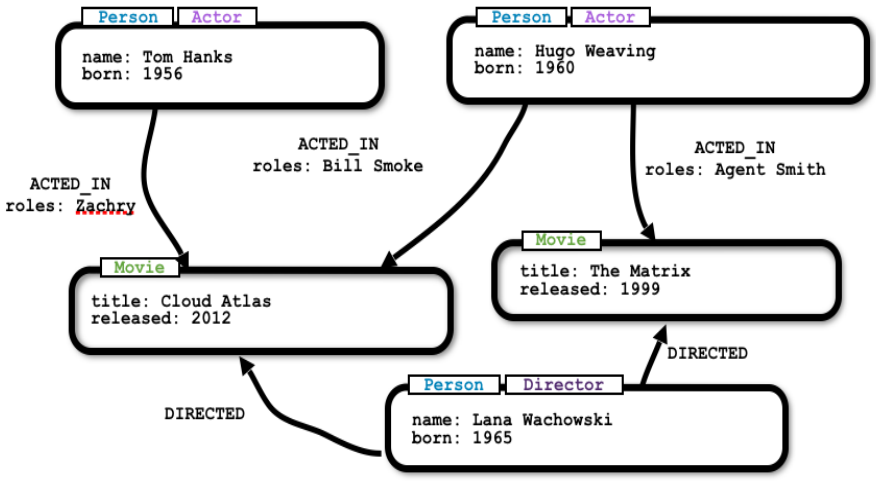


**Figure 2: Figure 6: Match node and relationship format of labeled property graph [17]**

Through queries, the graph data model can answer questions. By arranging a data structure for the graph database, it finds solutions to technical and business problems.

Graph data modeling is known as “whiteboard-friendly”. One can draw the model just as it is drawn on a whiteboard without model re-formatting which simplifies and visualize the model easily. Moreover, labels and properties for the model nodes and relationships can be added as shown in Figure 2.

#### 2.2.4.3 Entities of Graph Data Modeling

The next subsections will identify the graph data model’s main entities.

##### *2.2.4.3.1* Nodes

Nodes and relationships (which come later) are the two fundamental entities forming a graph. Nodes represent entities and other domain components with a unique conceptual identity. Name-value pairs of data can be stored in a node in the form of properties as well as roles and types in the form of labels [17].

##### *2.2.4.3.2* Labels

Labels are assigned to nodes to cluster and classify them into sets. Nodes that have a common label are in the same sets, which makes database queries easily and efficiently work with sets but not with the whole graph. Any number of labels including zero can be assigned to a node, so labels are optionally added to the graph. As nodes were identified through nouns, labels can be identified by generic nouns such as a group of persons, places, or things [17]. For example, if a graph has six nodes (user, activity, location, environment, temperature, and humidity), they can be labeled as (user {user}, user parameter {environment, location, activity}, and environment parameter {humidity and temperature}).

##### *2.2.4.3.3* Relationships

Two nodes can be connected, and related nodes can be found via a relationship. The relationship arrow is directed from the source node to the target node. A relationship must be stored in a particular direction; however, it can be quired without specifying the direction. A node cannot be deleted without deleting its relevant relationships due to the "No broken links" role. Relationships can be identified by verbs or actions in the model domain. Continuing with the running example, relationships can be identified as (the user has an environment, a user has a location, a user has an activity, humidity belongs to the environment, and temperature belongs to the environment) [17].

##### *2.2.4.3.4* Properties

Nodes or relationships can store name-value pairs of relevant data called properties that support most standard data types. Properties can be identified by knowing what kind of questions are asked of data based on the use case. Continuing with the running example, one can ask (What are the name, age, and weight of the user? what is the value of the humidity and temperature at a certain time?). Figure 4 shows the complete data graph with the labeled nodes, relationships, and properties assigned to them. This

model can be queried to answer the questions defined by the properties. The model can be easily updated as it is simple and flexible.

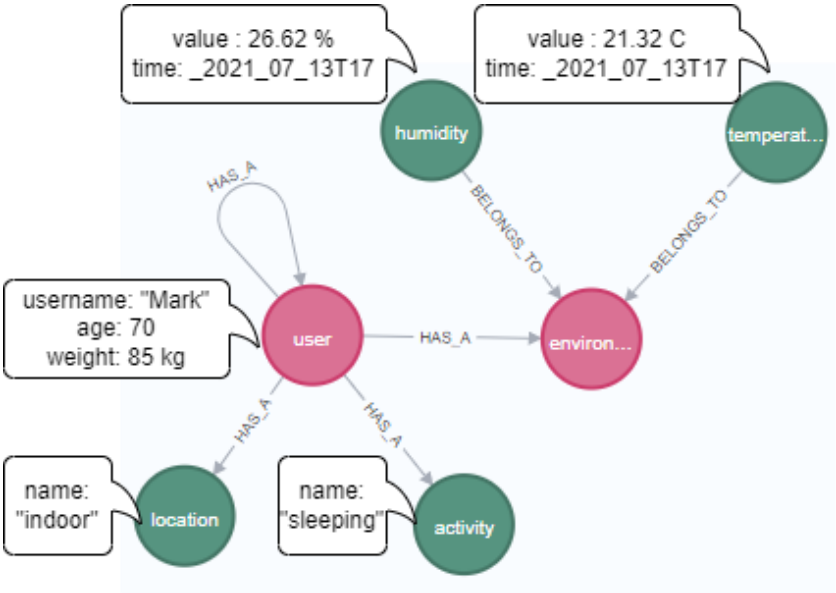


**Figure 3: Graph data model with properties added**

### 2.2.4.4 Designs of Graph Data Modeling

In this section a variety of modeling decisions by which graph data can be represented. As the way of constructing the data model affect queries and performance, suitable changes should be applied to find the best solution for the use case and achieve the best performance of the queries. Graph data models have no certain schema, so the data models adapt to a change easily with the problem specification. They can deal with the problem definitions changing over time and allow adjustment within part or entire of the graph with no effort. Developers are in the charge of defining the data model structure and entities for queries. The next sub-sections will discuss various ways of looking at various data sets and how each one affects queries and performance for traversing graph data [18].

#### *2.2.4.4.1* Property vs Relationship

An early aspect to be decided is if something is modeled as a node property or as a relationship to a separate node. For example, the property activity assigned to the user node is shown in Figure 4.

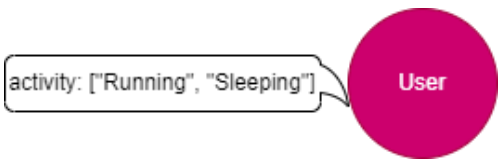


**Figure 4: A user node with activity property assigned.**

It is simple to know a user activity(s) via a query that finds the user node and returns a list with the activity property. However, a much more complex query would be needed to find out users that share activities as the query will search for the user node and

then traverse each of the activities in the property array and try to find a match with each value in the other user's property array of activities. That will affect performance and query complexity negatively due to the comparison of node properties and nested loops.

Another way to model the user activity as a system context element is to create users and activities nodes separately as shown in Figure 5. In this way, all users that share the same activity can be connected to that activity node. The query will be to find the user node, then find all nodes to which the user node relates to the HAS_ACTIVITY relationship. This way is much simpler as the graph pattern (entity-relationship-entity) is used to find needed.

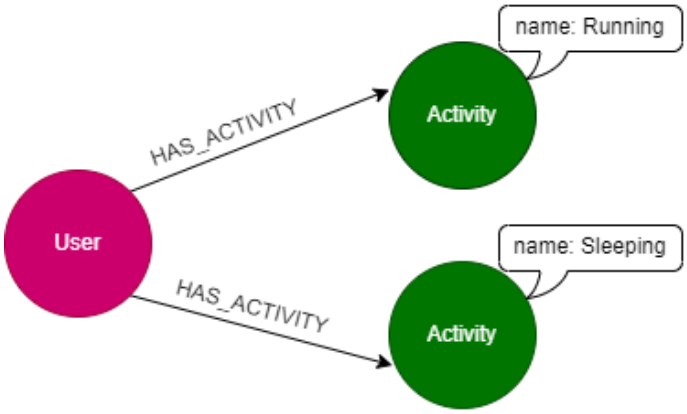


**Figure 5: The User and Activity entities as separate nodes.**

There is the no worse or better way to create the model, it only depends on the form of queries applied to the data. The first model suits the data analysis on the individual items and comes back with only details about the entity. The second model suits finding shared information between entities or groups of nodes.

#### ***2.2.4.4.2*** Complex Data Structures

Sometimes data models are not as such as simple. However, data needs to be highly organized, so patterns and decisions can be seen and taken respectively. Hyperedges are one modeling design that suits such complex models, where intermediate nodes are created to model relationships or connections between multiple entities at a point in time. For example, a university course has multiple offerings of the same course and instructor in the same building, which means that each offer is an instance of the course.

#### ***2.2.4.4.3*** Time-bound Data and Versioning

Data can be included in relationship type to model time-specific data and relationships. Query performance can be improved by specifying a date as a relationship type, so traversing only these dating relationships is optimized. For example, a model for airline flights with a particular flight on a specific day between two specific locations. Another similar model is versioning by which version of data can be tracked, data changes can

be shown, for example, with the aim of trend analysis. Multiple data models can be combined in one graph to take the advantage of each.

#### 2.2.4.5 Practical recommendations for Graph Modeling

From what has been discussed so far, it is obvious that there is no best data model. However, the data model designer chooses the most suitable data model based on the type of questions and queries applied to the model. Even if he has not determined what exact query syntax to use, at least he understands the goal of the constructed system. For example, if a specific date range of results is needed, the date should be stored as a separate node or relationship, but not a property on the node. As opposed, a model with a high-level hierarchy is more suitable for a university program domain to find efficiently all classes related to a certain topic [19].

##### ***2.2.4.5.1*** Prioritize Queries

Finding the most suitable model for every query is not a simple task. So, optimizing the performance on each separate query under resources, time, and code is the right methodology. For Neo4j as being flexible, the model can be adjusted with priorities changing over time.

The next sub-section will introduce cypher as the query language that will be used during the concept design and the prototype implementation phases.

#### 2.2.4.6 Cypher Query Language

Cypher is one of the query languages which is based on patterns. It is also designed to work with different versions of these patterns. This makes cypher an easy language to use as pattern recognition is one of the basics by which the brain works [20].

##### 2.2.4.6.1 Cypher Syntax

Cypher's construct is based on English which makes it visual, and easy to read and understand as was obvious in the example shown in Figure 3.

###### *2.2.4.6.1.1 Cypher Comments*

As with most programming languages, a cypher statement can start with // to indicate that this statement is commented. It is usually used to explain the syntax or the query functionality.

###### *2.2.4.6.1.2 Representing Nodes in Cypher*

As it was mentioned in 2.2.4.3.1, nodes are identified by nouns and objects in the data model. Nodes are surrounded in cypher with parentheses, e.g. (node), which is more like the circles visualizing the nodes in the data model.

###### *2.2.4.6.1.3 Node Variables*

Nodes can be referred to as variable (n), to use further in extracting the information stored in the node. Cypher also supports creating non-relevant nodes and referring to them with empty parentheses ().

*2.2.4.6.1.4 Node Labels*

As was mentioned in 2.2.4.3.1, node labels are assigned to group similar nodes. Cypher can be informed to check labels for specific information, which helps in entity recognition and query optimized execution.

*2.2.4.6.1.5 Representing Relationships in Cypher*

Cypher represents the relationships between nodes with arrows → or ←, which is like the visual representation. Relationships can also store properties as a square bracket inside the arrows. Cypher also supports undirected relationships represented with no arrows, but only two dash lines --**,** which means that it can traverse relationships in both directions.

*2.2.4.6.1.6 Relationship Types*

To classify relationships, types are assigned like labels classifying nodes. Naming rules are recommended for the relationship types as verbs or actions, so it is easy to read and write cypher queries, for example [:IS_FRIENDS_WITH] which makes sense between two nodes of persons.

*2.2.4.6.1.7 Relationship Variables*

Same as nodes, a relationship variable [r] can be created to be used further in queries. Cypher also supports creating a non-relevant relationship and referring to it with arrows →, ←, or dashed lines.

*2.2.4.6.1.8 Node or Relationship Properties*

As it was mentioned in 2.2.4.3.4, properties are described as name-value pairs added to nodes and relationships to store details. For properties, cypher uses curly braces within the parentheses or the brackets of a node and relationship respectively. Figure 7 shows examples of properties for nodes and relationships, which can be written, for example, node property: **(p: User {name: 'Mark'})** and relationship property: **- [r: HAS_A]→ (n: environment)**.

*2.2.4.6.1.9 Patterns in Cypher*

Graph patterns consist of nodes and relationships, which build together simple and complex patterns. Patterns, which represent the most powerful graph capability are written either as a continuous path or in the form of smaller patterns separated and tied together with commas. In cypher, patterns have the form of (entity-relationship-entity), for example: **(p:User {name: "Mark"})-[r:HAS_A]→(b:environmet {type: "External Conttext"}).** Some keywords discussed in the following section are used to inform cypher what to do with these patterns.

In the concept phase, the cypher queries will be designed after adjusting the system's current context model, so that the concept objective of mapping the system's external context to its dynamic behavior is met.

### 2.2.5 Derivation of a Modeling Approach

For further processing and learning, heterogeneous contexts should be joined uniformly in a model. When the (CP-AS) context is modeled, it must be system-centering. The complex structure and components relationship should be considered. Moreover, consistency checks and reasoning should be used because the context is uncertain. As formality and interoperability between systems are introduced, semantic techniques became important. Ontological modeling allows a formal description of the context, but the complex hierarchical structure and limited reasoning by First Order Logic (FOL) with a shortage of methods in learning from historic context and dealing with uncertainties. In addition to ontologies, fuzzy logic can add probabilistic modeling. Semantic or (Knowledge) Graphs consist of nodes and edges representing a flexible approach for modeling physical entities and relationships in an assigned graph. Extended expressivity beyond (FOL) is enabled by applying graph-based modeling. Graph analytics are used to learn and analyze similarities between created links. They are also easier to access and managed in a central node manner due to their plain structure [5].

### 2.2.6 Hybrid-Context Modeling for Cyber-Physical Systems

Figure 6 shows a context model for a (CP-AS). It also shows the system characteristics such as system interactions, and user-based and environmental quantities. As a graph base, LPG introduced in 2.2.4.2 is used to allow strong formality and dynamic learning.

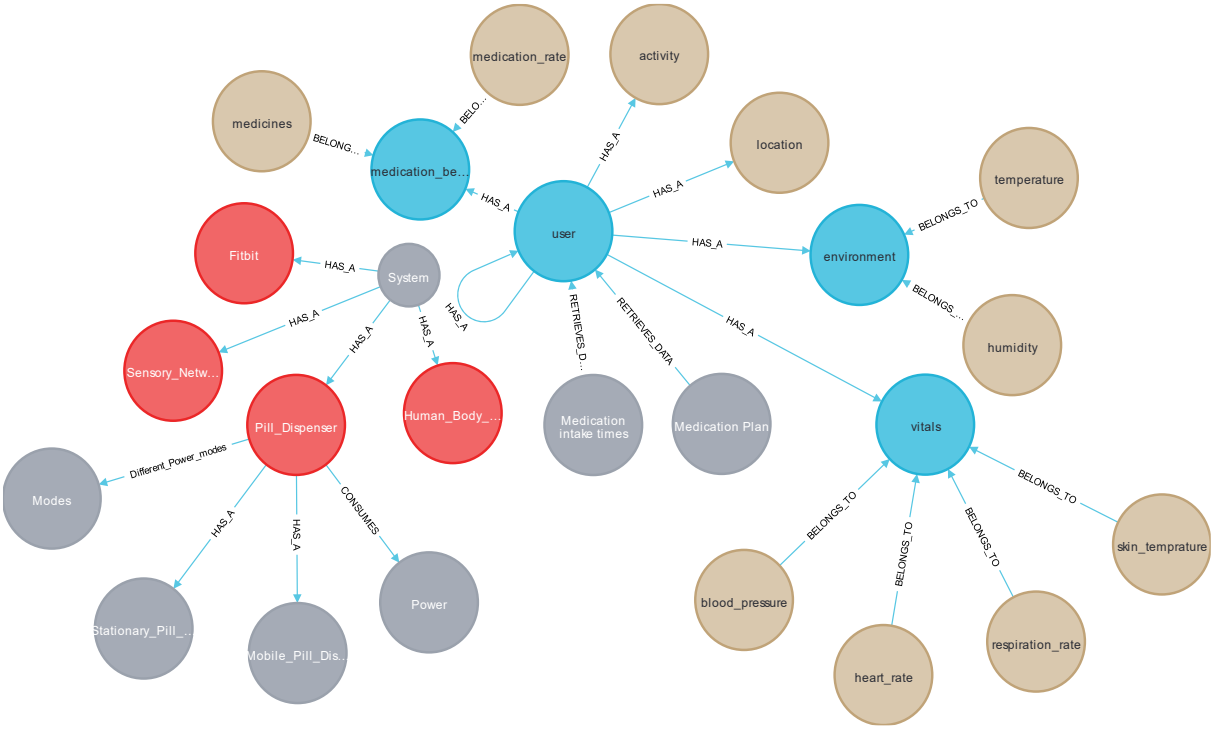


**Figure 6: Current context model representing the system, user, and environment [21]**

## 2.3 Automated Pill Dispenser (Context Modeling Use Case)

As a use case for (CP-AS), Figure 7 shows the context modeling of an automated pill dispenser.

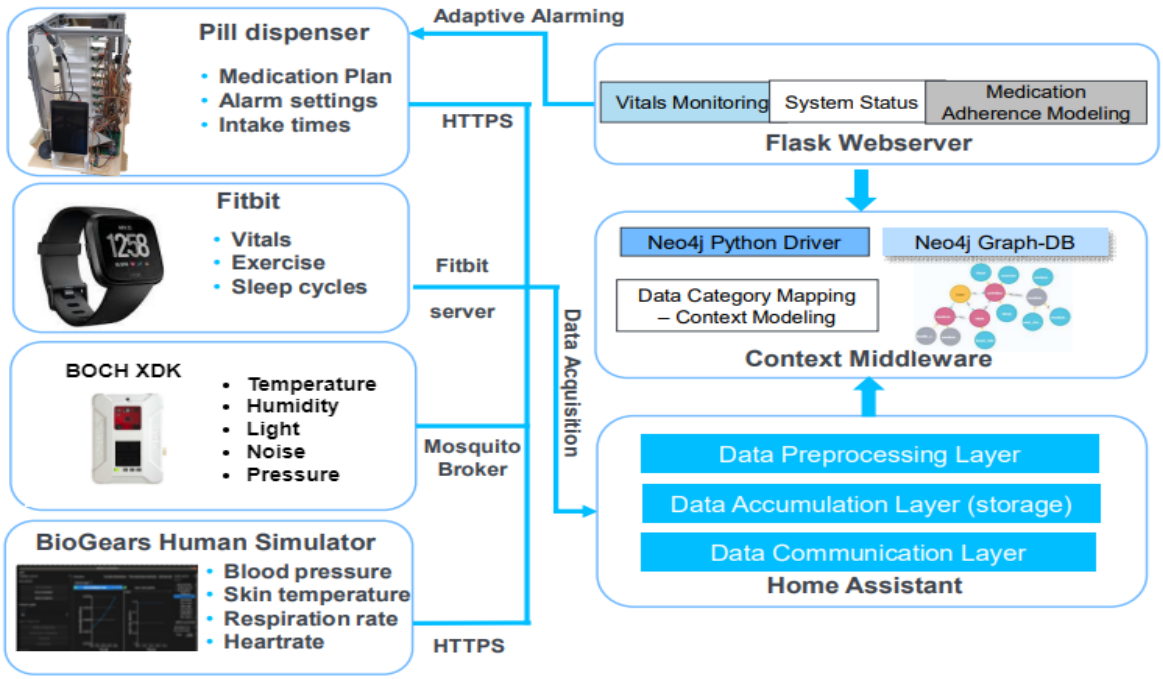


**Figure 7: Context-aware medication assistance system [3]**

### *2.3.1* Pill Dispenser Description

The pill dispenser is a mediation system assisting old people to manage pill filling, reminding them, and tracking their medication dosage due to the increased number of prescribed medications with the age [22].

The system consists of a smartwatch, a microcontroller with motion and air quality sensors, and a pill dispenser with patient medication data. These system components are the three data sources representing the system, the user, and the environment. The system middleware is built on an IOT integration platform supporting adaptive alarming and medication process modeling. The model keeps the system consistent and correlates the attributes of the user and the system usage [5].

### 2.3.2 Pill Dispenser IoT Architecture

The pill dispenser was built on IoT-Approach requirements to which the derived system requirements. These requirements were defined during the system design phase. The system architecture consists of four main components that can be described as the following [23]:

**Stationary pill dispenser** is responsible for pill box management, pill dispensing, and pill intake detection. It communicates with the cloud server to retrieve updated data.

- **Mobile pill dispenser** is responsible for daily pill box exchange and pill intake via box opening. It communicates with the mobile app via Bluetooth to exchange box opening times and reminder settings.
- **Mobile app** is responsible for scanning medication plans via QR-code to generate pill-filling instructions and alarms. It allows medication overview and personal alarm setting.

The next sub-section will introduce some of the pill dispenser functions that will be used as use cases in the conception of Chapter 3.

## 2.3.3 Pill Dispenser Functions

As an automation system the pill dispenser has certain tasks or functions that can be described as the following:

### 2.3.3.1 Alarm Function

This function is responsible for giving an alarm signal in the form of an audio sound functioned by a speaker module and blinking LEDs when the system time reaches the alarm time set by the user. As shown in Figure 8, the daytime is divided into four periods (morgen, noon, evening, and night). The user can set an alarm for each daytime every day during the week. The user can also choose the alarm type if it is by audio sound, and/or by LEDs.

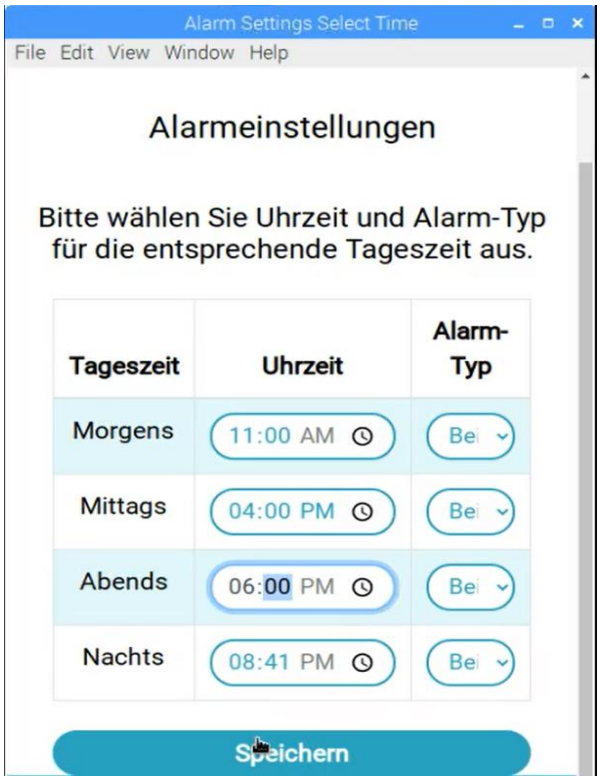


**Figure 8: Pill Dispenser Alarm Function**

#### 2.3.3.2 Motor Positioning Function

As shown in Figure 9, the stationary pill dispenser has eight magazines for the seven days of the week and one extra spare magazine in which the pills are stored and organized. There is an LED attached for each magazine. Around one minute before the alarm time, the pill dispenser motors (one vertical motor and one horizontal motor) are moved to the related dispensing position (weekday, daytime) after being calibrated by reaching their mechanical limit switches, as a reference point to their current position.

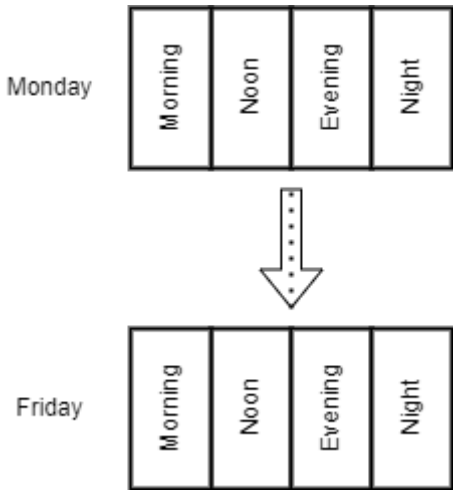


**Figure 9: Pill Dispenser magazines**

#### 2.3.3.3 Pill Dispensing Function

After the alarm is positioned in the correct (weekday, daytime) position, the system waits for a user confirmation via the system user interface indicated in Figure 10, so the pill dispensing function starts.

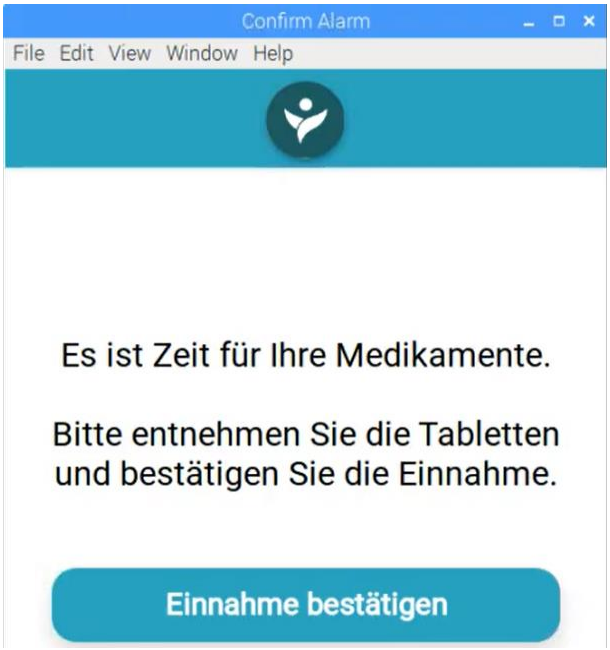


**Figure 10: Pill Dispenser alarm confirmation message**

The system sends an extraction signal to an electro-mechanical solenoid to push the magazine closed contact, so it is opened, and the pill falls and is detected by a PIR motion sensor. It detects the pill dispenser fall and the removal by the user.

The next section will introduce graph data modeling as a modeling approach for the system’s context.

## 2.4 Concluding Remarks

A pill dispenser discussed in 2.3, which serves as the project's initial starting point, is depicted as a labeled property graph (LPG) in Figure 6. This graph depicts the system as well as the external context (such as the user, environment, or other systems' parameters). The system representation displays the elements (structure), functions (inputs and outputs of the control program, and operations within the system), and control software). The project aims at learning how the external context affects the system while it is in use. For instance, how the environment (such as heat or air pollution) can impact the behavior of the entire system. This can be achieved by modeling (mathematically) the discrepancy between the expected and actual system behavior, which is an illogical or unexpected leak of compatibility or similarity using the FBS framework introduced in 2.2.2, and then attributing (relating the causes of) this discrepancy to the current semantic (model) and temporal context. Currently, Neo4j graph databases introduced in 2.2.4.3 is used as a tool, which provides, Python interfaces, and graph algorithms [19]. Then the concept can be demonstrated using a simple use case example (single sensor or actuator). For example, the user target response time to the system alarm can be compared to the actual user response time affected by the environment noise level.

# 3 Conception

## 3.1 Overview

The pill dispenser has a structure including its hardware components such as sensors, actuators, and microcontrollers, and software components such as control code, communication, interface, and user interface. It also has system functionalities such as adaptive alarms, motor positioning, and pill dispensing. The idea here is to develop a model for the contextual elements that might affect the system's internal dynamics at its operating time. The following next sub-section will introduce the system meta model and the purposed meta model adjustment.

### 3.1.1 Pill Dispenser Meta Model

Figure 11 shows the meta model of the pill dispenser. It already existed, and it is still being developed and optimized by another research colleague to improve the model query performance. It is defined as the static part of the system's context model. It includes all the static nodes representing the user context such as user, user activity, user location, vitals, and environment. It also contains the system's context such as the stationary and mobile pill dispenser, the hardware, and the software components. In this work this meta model is intended to be adjusted based on the FBS design framework introduced in 2.2.2, to adapt this design model during the system runtime, as it will be introduced in the concept developed in the sections of this chapter.

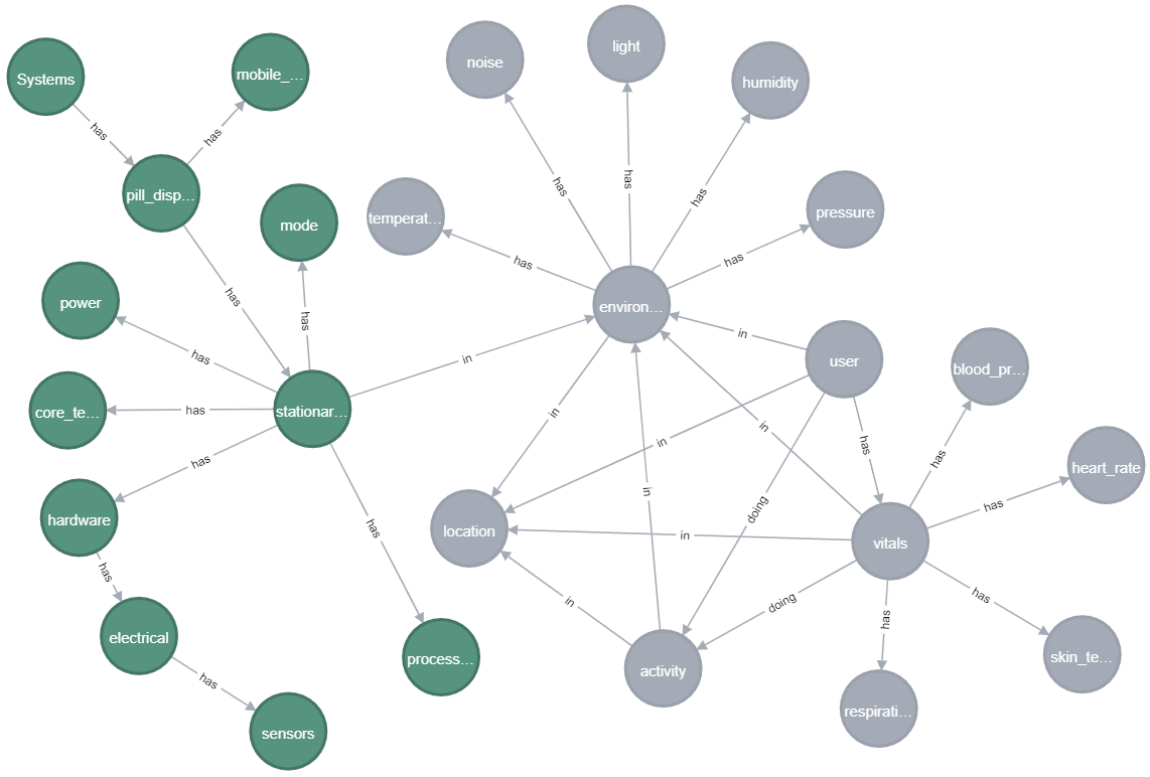


**Figure 11: The Pill Dispenser Meta model**

The development process of the concept will require three working steps:

- First an adjustment of the current meta model by adding labels for the different model entities to categorize them, so they either represent a system structure, function, or behavior. New relationships between model entities will be added to relate how some entities are affected by other entities.
- Second, developing a query procedure of how the context model is accessed to models detect the output discrepancy of the system functionalities.
- Third, developing and implementing use cases by which the developed concept is validated.

The next three sections will address these working steps.

## 3.2 Meta Model Adjustment

All nodes of the meta model are to be labeled, so each node is either categorized into function, behavior, structure, internal context, or external context as was proposed previously in the basic documents. Figure 12 shows the proposed updates for the user's context and the system's context respectively. This update will help to map the system model to the Functional-Behavioral-Structure (**FBS**) framework also introduced in the basic document. This FBS will be used further to extract the system's actual behavior (**Bs**) extracted from the system structure (**S**) and compare it with the system's expected behavior (**Be**) extracted from the system functions. Figure 12 considers the user context, the environment and user are considered external environmental and user context entities outside the system boundaries, so they are labeled as **External Context**. Figure 12 also considers the system context inside the system boundaries so they are denoted as **Internal.** The system hardware components are considered system structures denoted with **Structure**, while each of these structures has some features considered as system functions **Function**, for example, the motor has a motor positioning function. In the same concept, the dynamic values of the output of these functions are considered as the system's actual behavior denoted with **Behavior.**

Based on the use case introduced in section 3.6 there new nodes will be added to represent the system's expected behavior affected by the system's internal context and external context which in our case are the system internals, environment, and user.

In addition, a new relationship "AFFECTS" or "INFLUENCED_BY" will be added to the graph going from the nodes representing the external contextual elements such as environment and user elements to the nodes representing the system functions and internal behavior. For example, the (External Context –AFFECT→ Function) relationship indicates that a system context affects a system function. In the other direction (Function –INFLUENCED_BY→ External Context) relationship indicates that the output of a system function might be influenced by an external context.

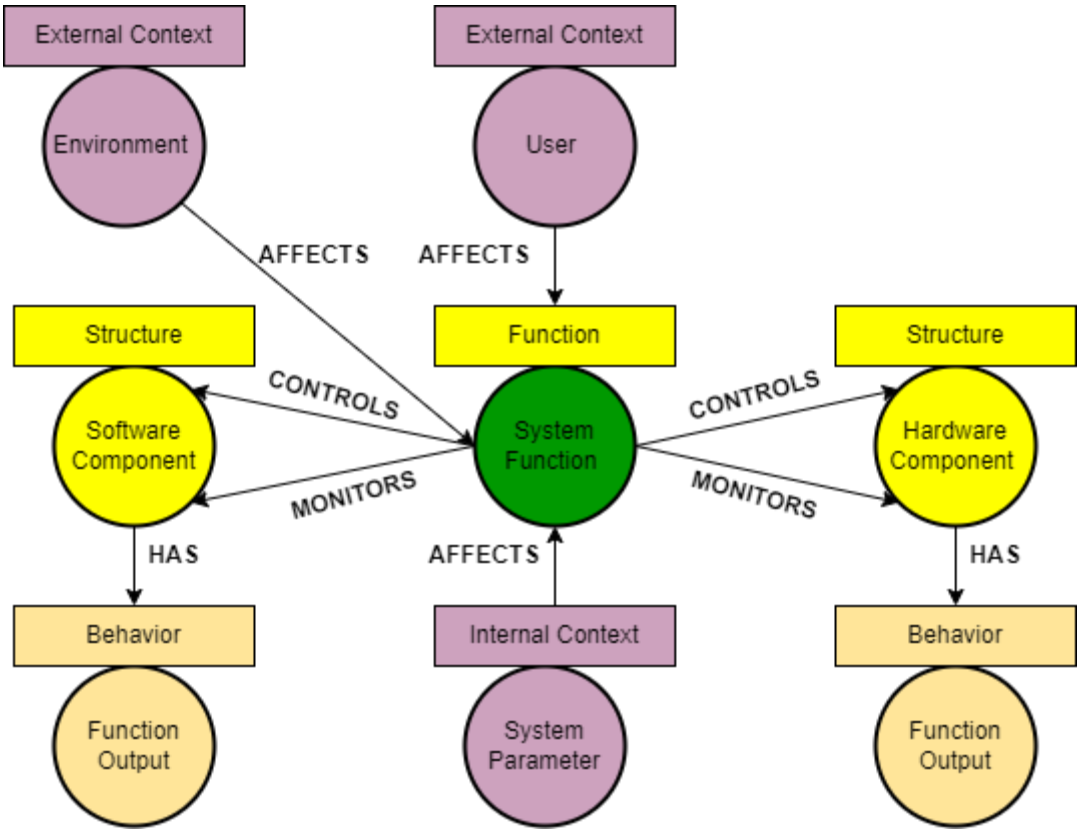


**Figure 12: Pill dispenser FBS Mapping**

Another two relationships "CONTROLS" and "MONITORS" will be added to the graph going from the system functions to the system structures to identify which function controls or monitors which structure. For Example, the (Alarm –CONTROLS → Speaker) relationship indicates that the alarm function controls the output of the speaker which is its sound level. (Pill Dispensing -MONITORS → PIR) represents that the pill dispensing function monitors the PIR motion sensor in case of failure in detecting pill fall after alarm confirmation by the user.

Another adjustment that will be done is to add new nodes representing the system functionalities such as (Alarm, Position motors, and Pill dispensing) and their dynamic behavior such as (alarm confirmation times, motor position times, and pills detection times) that do not exist in the current meta model graph to represent the system use cases discussed in 3.6.

## 3.3 Meta Model Query Procedure

### 3.3.1 Query Flowchart

Sub-section 2.3.3, has introduced the stationary pill dispenser's different functions. Each of these functions can be modeled in the system meta model as a function node that has input nodes, and output nodes, and is affected by context nodes. The function can then control a structure node.

Figure 13 shows the model query procedure flowchart. The input **X** of the function under consideration is queried. Based on the function and the inputs, the function's expected output **Ye** is calculated so it can be considered as a reference baseline to the function's actual output **Ys.** Then, the actual output **Ys** is measured and compared with the function's expected output. In case of out discrepancy, the dynamic context **C** is queried and mapped to the output discrepancy.

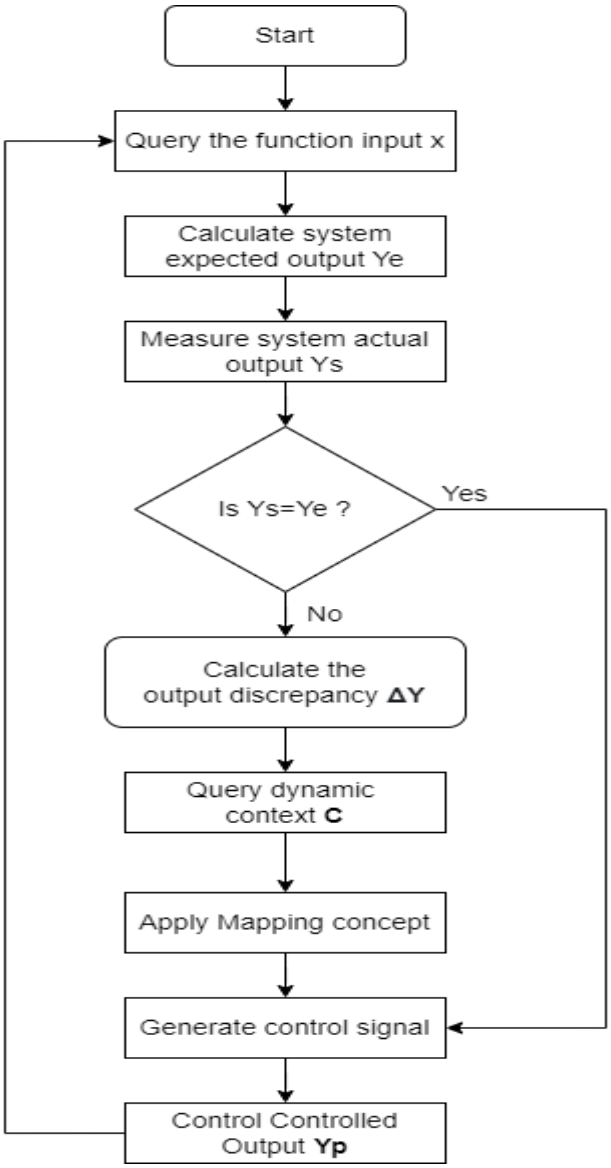


**Figure 13: Query procedure flow chart**

### 3.3.2 Query Design

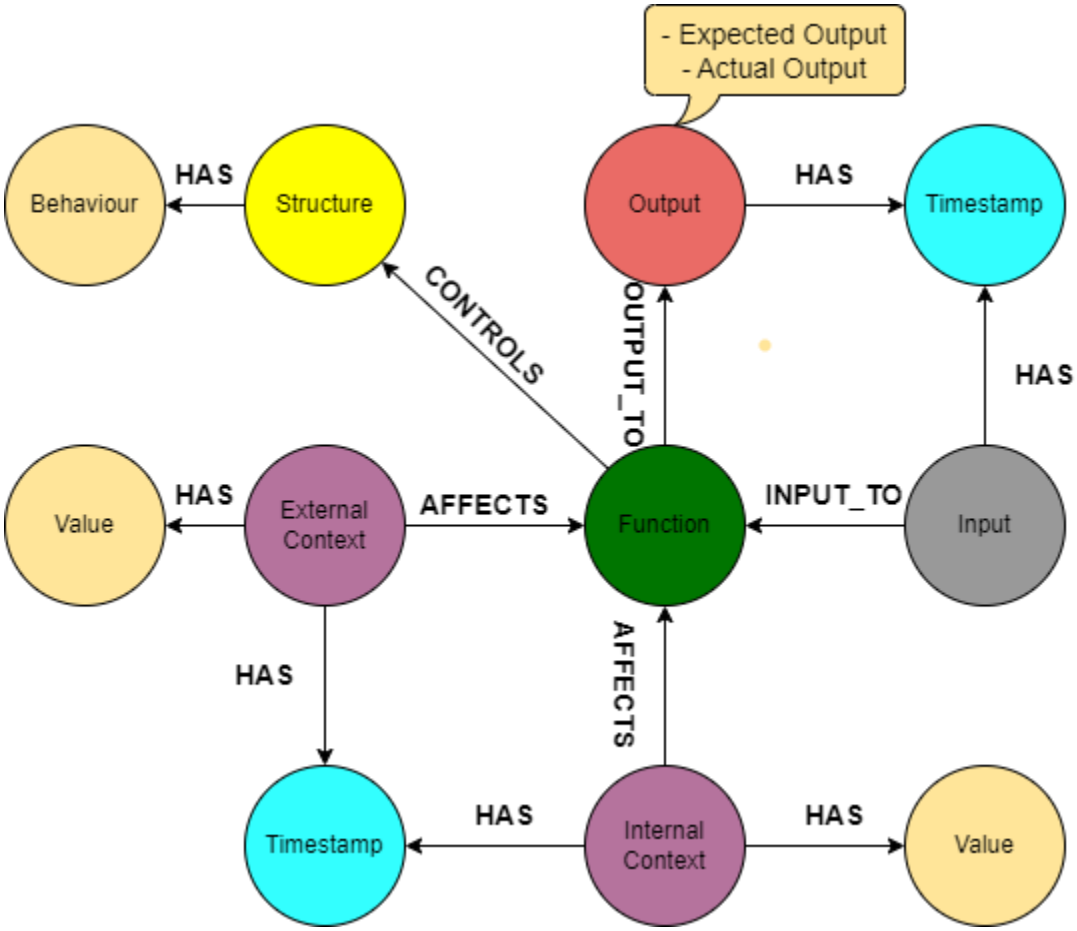


**Figure 14: Query Design Diagram**

Figure 14 shows in the middle a system function affected by some internal and or external context. Each context node has a timestamp and value also represented as nodes. This function has input and output nodes with expected and actual output denoted as key property values. A function node controls some system structure that has some behavior.

## 3.4 Dynamic Context Mapping

Based on the introduced concept, the dynamic context affecting the system during runtime can be mapped in two methods introduced below in this section. The goal of this context mapping process is to have an X→Y relationship between the system's external context and its output discrepancy, so the reason behind this discrepancy is defined and it can be controlled to derive the system again to behave as it is expected.

1. Mapping mathematical equation

For context elements with continuous values such as noise or light in the alarm use case, the system's output discrepancy value can be to the related context values as a mathematical equation:

$$Y = [\mathrm{B}] \times [\mathrm{X}]$$

where the **X** row vector is the context vector, while **the B** column vector is the coefficients for the equation. The coefficient vector can be estimated based on the dataset previously recorded via the coefficients estimate command supported via MATLAB environment after the data is queried from the context model and normalized, so they are unitless.

2. State Machine

For the context elements with discrete values such as user location or user activity in the alarm use case, a state machine can be designed to relate the system's context values to its discrepancy value. For example, if the user was resting and his location was indoors what was his output discrepancy currently? Then for each state with a certain context combination, this discrepancy value is calculated over the entire dataset during the system's 1st round (learning round). In the system 2nd round (control round), it can estimate the system discrepancy as the average of the values calculated during the learning round.

Example for the stationary pill dispenser assuming the user is:

System during the 1st round:

**Table 1: Mapping Process as a state machine**

| timestamp | daytime | User Location | User Activity | Output Discrepancy |
|---|---|---|---|---|
| 24_07_2021 | morning | Indoor | sleeping | +45 minutes |
| 24_07_2021 | noon | Indoor | resting | +05 minutes |
| 25_07_2021 | morning | Indoor | sleeping | +15 minutes |
| 25_07_2021 | noon | Indoor | resting | +15 minutes |

System during the 2nd round:

State (morning, indoor, sleeping) -> Learned Discrepancy = (45+15)/2 = 30 minutes.

State (noon, indoor, resting) -> Learned Discrepancy = (05+15)/2 = 20 minutes.

## 3.5 System Model

### 3.5.1 Software System Architecture

Software architecture is the discipline used to create these structures and systems as well as the basic components of a software system. Each structure is made up of software elements, their connections, and the attributes of both the elements and the connections.

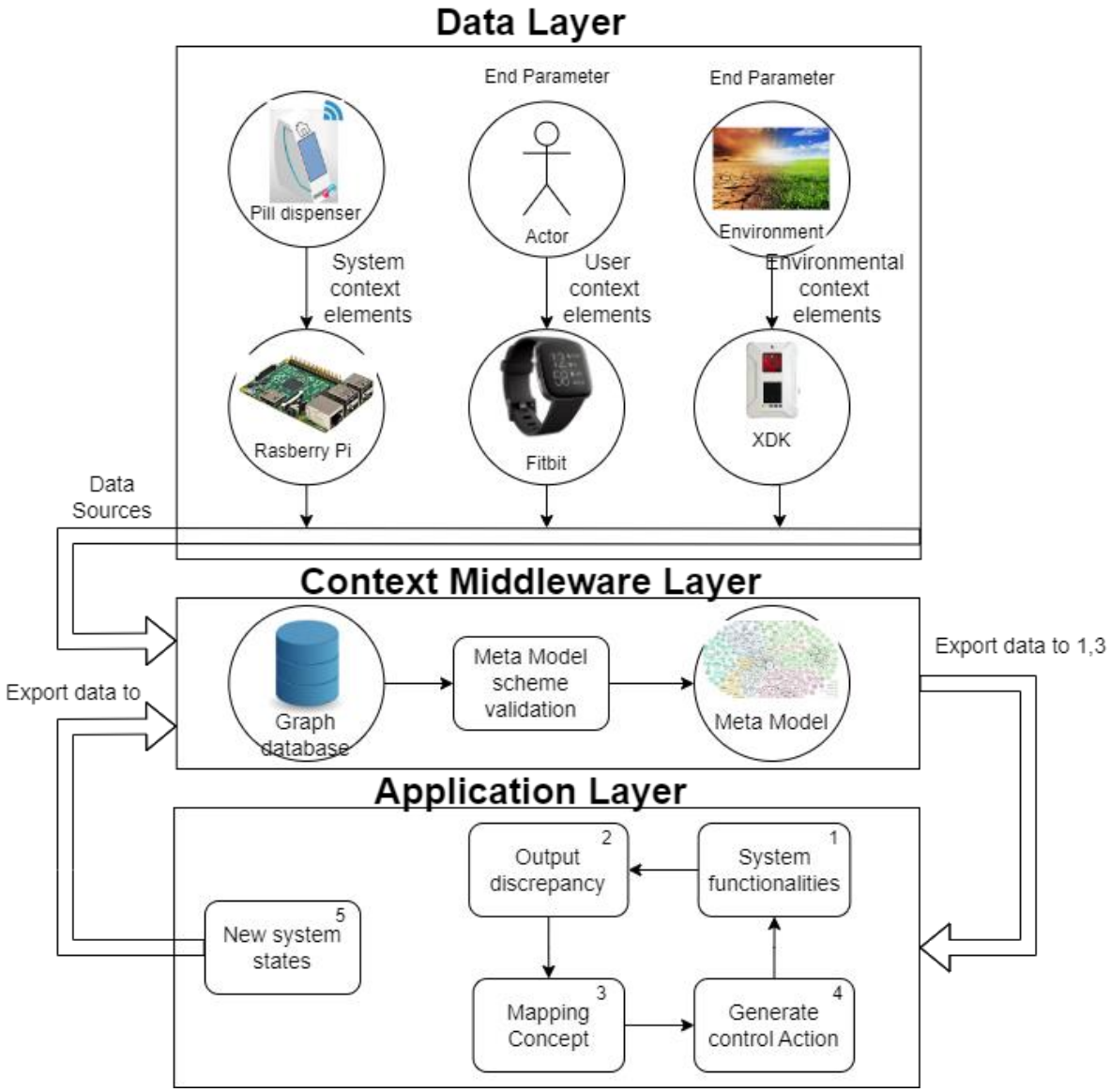


**Figure 15: Software System Architecture**

As shown in Figure 15, the software system architecture has three layers which are already existed in the system's previous implementation, however, in this project, new software components will be added as a part of the implementation introduced in Chapter 4. **The data layer** in which we can see that the system has three data sources. First, Fitbit is used to collect the user parameters such as vitals, daily activities, and sleep cycle. Second is the XDK used to measure the environmental context elements

such as noise, light, temperature, and humidity. The third is the pill dispenser storing the system's contextual elements.

**Context Middleware** in which the data is stored in the system database and then goes through a pipeline scheme that processes the different datasets does the data labelling and validates them before populating them to the system meta model. **Application Layer** in which the context and input data for the system functionalities will be quired from the system meta model. Then, it will be used to assess the external's system context on its system functionalities. Then, the output discrepancy is mapped to the dynamic context affecting the system. Then, a control action is generated to control the function output, so the system behaves as expected. Then, the new system states can be pushed back to the system database to be a part of the system dynamic context model.

## 3.5.2 Software Components

### 3.5.2.1 Data Processing

This software component is currently developed by another colleague to process the data coming from the three sources introduced in 3.5.1. It creates the structured datasets validated by their corresponding schemas before being populated to the structured meta model.

### 3.5.2.2 Expected Output Calculation

This software component is to be developed as a partial implementation of the alarm use case introduced in 3.6.2.1. It queries the previous alarm confirmation times stored in the context model and calculates the average confirmation time and returns it to the alarm function to be compared with the function's actual output.

### 3.5.2.3 Actual Output Measurement

This software component is to be developed as a partial implementation of the alarm use case introduced in 3.6.2.1. It queries the medication times from the context model and extracts the user's real confirmation time. Then it returns this difference in time as the actual output of the alarm function.

### 3.5.2.4 Mapping Criteria

This software component is to be developed to relate the dynamic context affecting the function to its output discrepancy. In case there is a discrepancy in the output, it queries the dynamic context stored in the context model and maps it to the discrepancy either mathematically or in a switch case based on the type of the dynamic context if it has continuous values such as noise and light intensity, or if it has discrete values such as user activity and location.

### 3.5.2.5 Output Control

This software component is to control the output of the system functionality based on the output of the mapping Criteria introduced in 3.5.2.2. It is supposed to be a simple

control logic such as a P controller to control the alarm output which is the speaker sound level and LEDs brightness based on one dynamic affecting contextual element. Interface Definitions

#### 3.5.2.6 Neo4j with Python Interface

Between each system software component, a Python interface is developed using the query cypher language introduced in section 2.2.4.6 so interactions between the system control codes and its context model are well structured and processed.

## 3.6 Use Cases

In this section, the general concept of addressing three system functions (Alarm, Position Motor, and Pill Dispensing) introduced in 2.3.3.3 is discussed.

### 3.6.1 Use case Sequence Diagram

As shown in Figure 16, the use case is mainly addressing a certain system functionality with an input vector **X** queried from the system context model which stores the system's external context (user, and environment) and internal context (system).

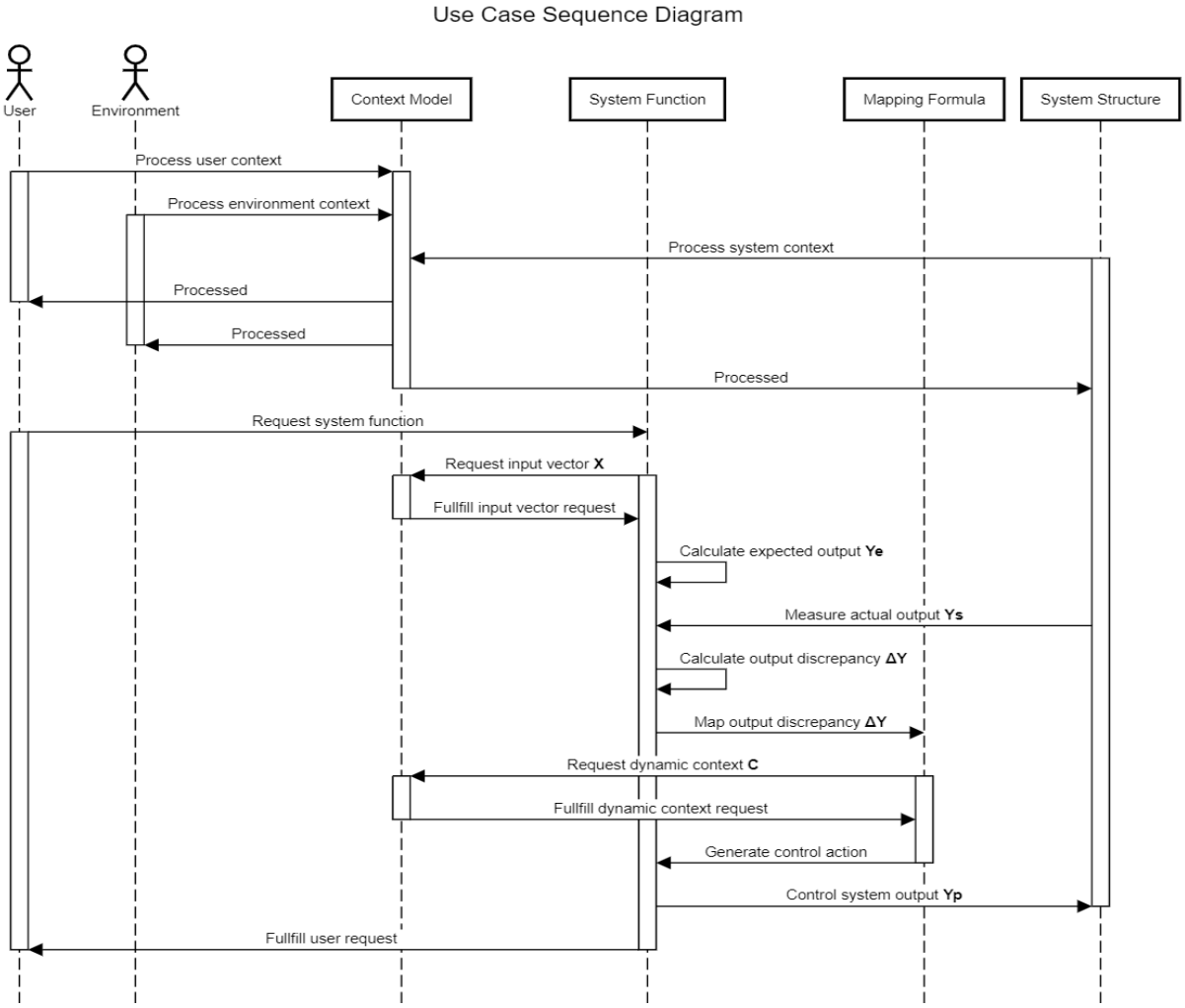


**Figure 16: Use Case Sequence Diagram**

The system functionality should output the system's actual output **Ys** driven from the system structure. Then this actual output is compared with the system expected output **Ye** driven from the system functions**.** The output discrepancy **ΔY** is mapped to the present dynamic context **C**, so next time when the system is present in the same conditions its output is controlled so the system is likely to behave as expected.

In the following subsection, the concept will be described in detail based on the system adaptivity use case and the alarm as a workout example. The further two subsections will investigate the same concept for the other use cases and workout examples.

## 3.6.2 System Adaptivity

As it was introduced in the basic document, the system is currently represented with a context graph in which the pill dispenser is represented as a user-centric cyber-physical medication assistance system. The system, user, and environment are represented via three data resources which are a Fitbit smartwatch, a microcontroller receiving input sensory data from motion and air quality sensors, and the pill dispenser storing the user's medication data. These data sources were used to create the context graph within the middleware based on IoT integration. As shown in Figure 7 the system adaptability can be achieved by quiring certain context nodes stored in the meta graph as modeled entities with their relationships [5].

### 3.6.2.1 Alarm

In Figure 17, in the middle, the Alarm (Speaker/LED) **Old function F** already exists in the system is shown. The following blocks were defined:

**Input X:** The User's medication times.

**Outputs Y:** They were classified into three categories:

- **Actual Output Ys:** The User Confirmation Time (Time difference between the Alarm started and user confirmation).
- **Expected Output Ys:** The Average User Confirmation time is calculated based on the previously recorded values for alarm confirmation. These values are previously stored in the context model, so they are quired and the average is calculated based on the average formula: Average confirmation time = **Σ Plausible Confirmation times / Number of Plausible Confirmation times**
- **Controlled Output Yp:** The Speaker level sound and the LEDs brightness are controlled so the Alarm is likely heard and seen by the user.

During the system being in the **1st round** (learning round), the **Output Discrepancy ΔY** is calculated and mapped to the **Dynamic Context C** presented at the time in which the output was not as expected and queried from its **Context model**.

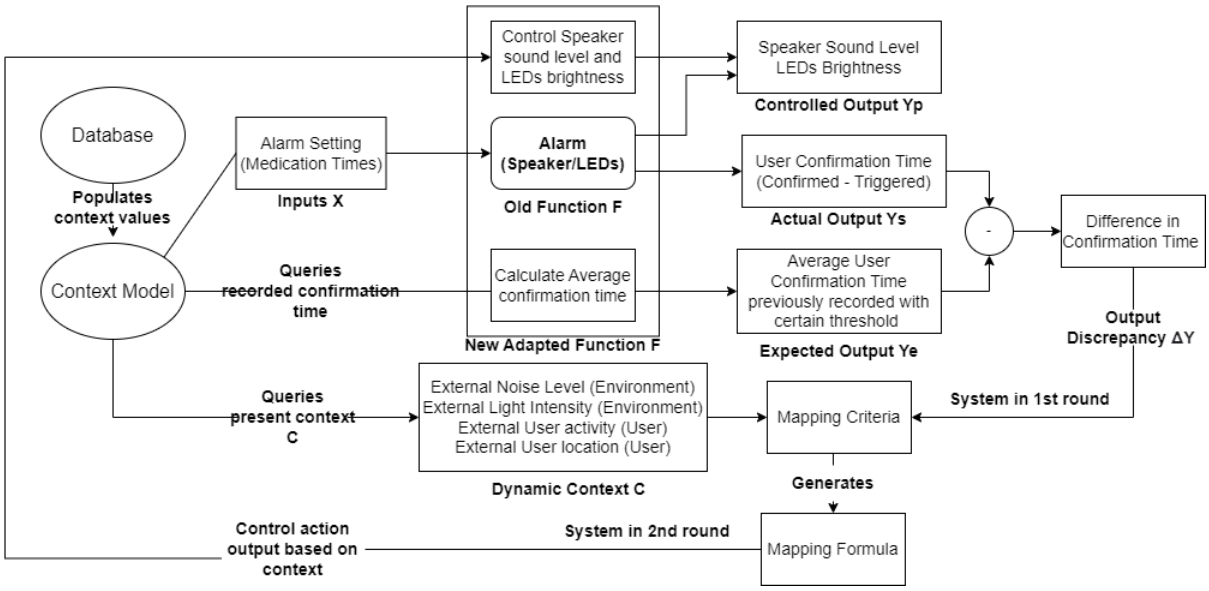


**Figure 17: Alarm Use Case Diagram**

In the alarm case, there are four external contextual elements **C** hypothesized to affect the alarm function.

- **Noise level**s coming from the environment might affect the alarm functionality. As the noise might interfere with the alarm sound, the system should adapt so the alarm sound can reach the user.
- **The light intensity** of the surrounding environment might affect the LEDs' brightness. For example, if the surrounding environment is too dark, the LEDs' brightness should adapt to this situation by giving a higher suitable brightness so they are seen by the user, especially for elderly people who might have eyesight weakness due to age factors. On the other hand, if the surrounding intensity is too bright, the LEDs should adapt this by decreasing their brightness so the system can save much energy.
- **User activity** might affect the user's ability to hear the alarm sound level output. For example, if the user is sleeping, the system should adapt such a case and try to give a louder alarming sound so that the user can hear and wake up.
- **User location** might affect the user's ability to hear the alarm sound level output or to see the LEDs. However, for use case simplicity it is assumed for the stationary pill dispenser that the user is always indoors.

Depending on the selected use case the **mapping criteria** could be a mathematical equation or based on a switch case. For the alarm use case, it can be a mathematical formula relating noise and light intensity to time output discrepancy. And a switch case relating the User location and activity to the output discrepancy. The generated **Mapping Formula** can be used during the system in the **2nd round (**control round in which the system is aware**)** to control the function output **Yp** (here will be the Speaker sound level and LEDs Brightness) so the Alarm can be likely heard and seen by the user. This newly added logic is gathered with the Old **Function F in** one **New Adapted Function F** via interfaces with the **Control code** and **Context model**.

### 3.6.3 System Monitoring

System monitoring is one of the essential factors in achieving high performance for software and hardware. Constant monitoring of the system is required to manage the software and hardware applications which leads to high-speed response for the external system inputs and internal states especially when it comes to real user interactions [24]. For the pill dispenser as a medical application, system monitoring is essential to manage the system functionalities related to the user's real-time experience. The next two sub-sections will introduce the motor actuation time and signal processing as workout examples in which system monitoring is involved.

#### 3.6.3.1 Motor Position Time

It is defined as the time difference between the motors being calibrated in their initial position till they are positioned Infront of the target magazine related to one weekday and daytime. This time factor might affect the user experience of getting the pills out of the system as fast and in accurate positioning as possible.

In Figure 18, in the middle, the position motor (stepper motors) **Old function F** already exists in the system is shown. The following blocks were defined:

**Input X:** The User's medication times.

**Outputs Y:** They were classified into three categories:

- **Actual Output Ys:** Motor's actual positioning time (Time difference between the motors calibrated and positioned).
- **Expected Output Ys:** The Average positioning time is calculated based on the motor's path. Based on the current implementation the dispensing point (the solenoid tip) is moving on the path indicated as a dotted line in Figure 19. As both motors are moving with the same rotational speed. The solenoid tip is moving in a path consisting of two straight lines. In case 1 at the left, the path of the horizontal step motor (x2-x0) in blue is **shorter** than the vertical step motor path (y2-y0) in red, so the solenoid tip moves on an inclined 45 ° straight line from (x0, y0) to (x1, y1), then vertically up till it reaches the positioning point (x2, y2). In case 2 at the right, the path of the horizontal step motor (x2-x0) in blue is **longer** than the vertical step motor path (y2-y0) in red, so the solenoid tip moves on an inclined 45 ° straight line from (x0, y0) to (x1, y1), then horizontally up till it reaches the positioning point (x2, y2). In case 1, the motor expected position time is the time of the vertical motor. However, in case 2 the motor expected position time is the time of the horizontal motor.
- **Controlled Output Yp:** The motor's rotational speeds are controlled so the Motors are positioned accurately in their expected time.

During the system being in the **1st round** (learning round), the **Output Discrepancy ΔY** is calculated and mapped to the **Dynamic Context C** presented at the time in which the output was not as expected as calculated.

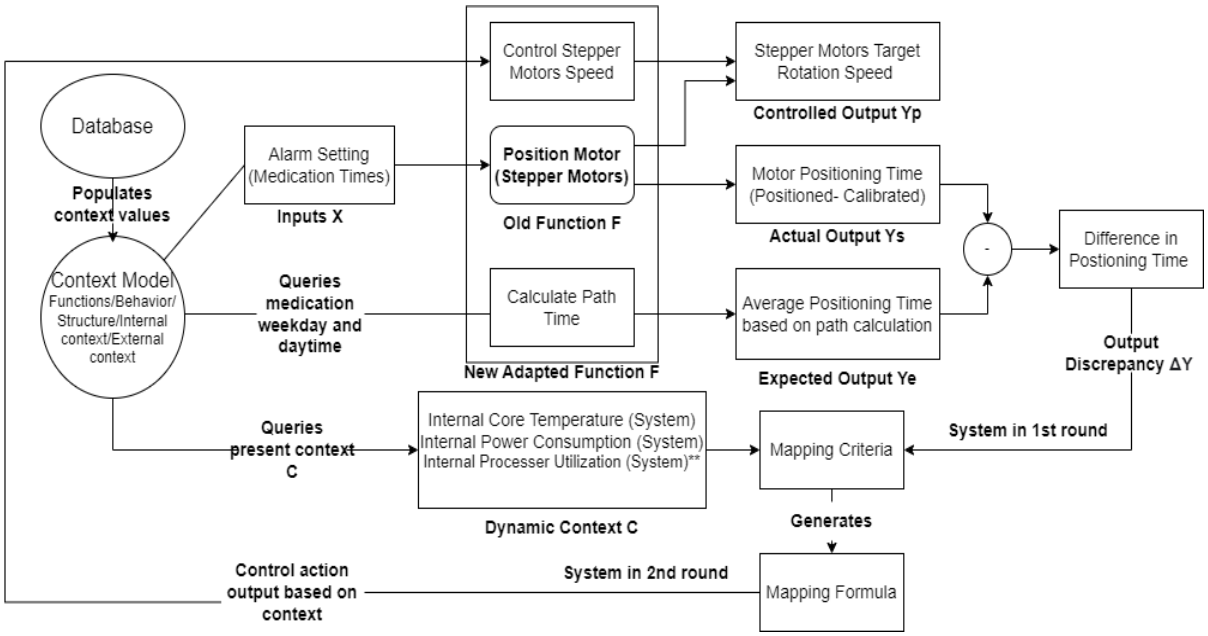


**Figure 18: Motors Positioning Use Case Diagram**

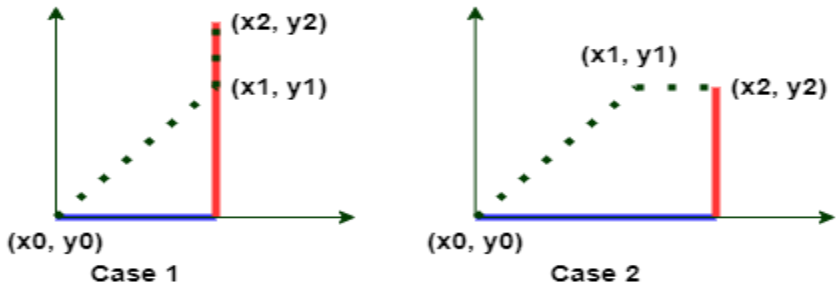


**Figure 19: Step Motors Positioning path**

In the motor's position use case, there are three external contextual elements C hypothesized to affect the alarm function.

- **Internal Core Temperature** might affect the system's performance in general. Raspberry Pi usually runs 20-30°C above the ambient temperature of the environment they are in. If Raspberry Pi is pushed to its CPU maximum temperature, it starts to perform slower than normal to prevent any damage [25].
- **Internal Power consumption** might affect the performance of the motor. It is defined to be the total system power consumption of the Raspberry Pi, sensors, actuators, controlling PCBs, and LCD. This total power consumption value might affect the motor's power consumption which furtherly affects its performance.
- **Internal Processor utilization** might affect the performance of the motor. As in the case of core temperature, increasing the processor utilization leads to an increase in the Raspberry Pi core temperature, so it starts to perform slower to prevent any damage.

### 3.6.3.2 Pill Dispensing Time

It is defined as the time difference between the alarm being confirmed by the user and the pills being detected by the motion sensor. This time factor might affect the user experience of getting the pills out of the system as fast and in accurate positioning as possible. It also might indicate a system failure, in case of alarm is confirmed and the pills are detected.

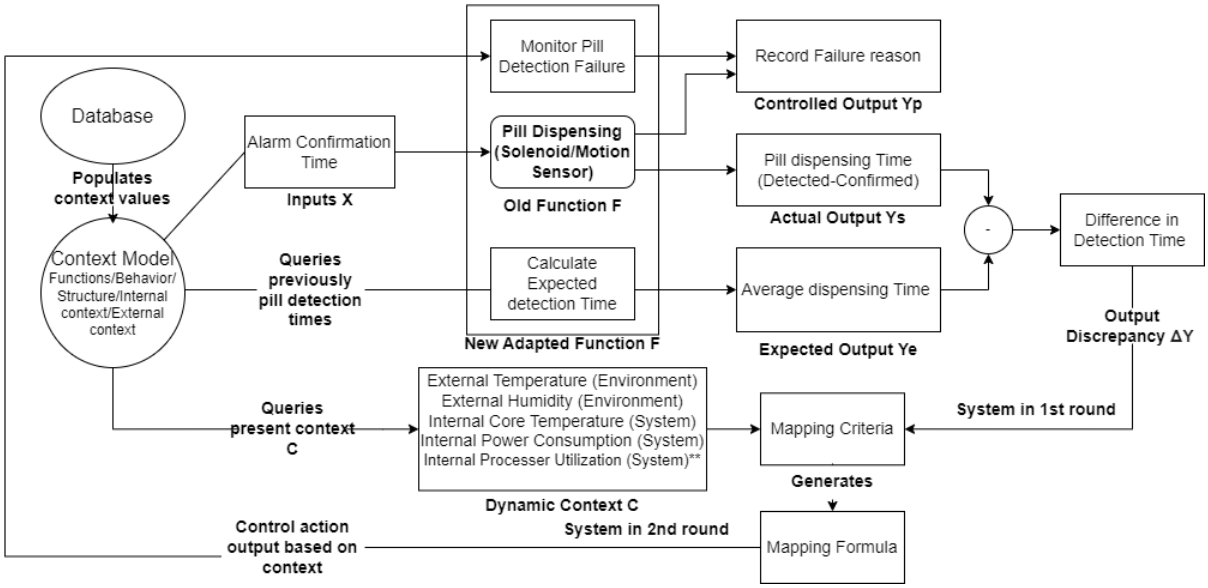


**Figure 20: Pill Dispensing Use Case Diagram**

In Figure 20, in the middle, the pill dispensing (solenoid/motion sensor) **Old function F** already exists in the system is shown. The following blocks were defined:

**Input X:** The Alarm confirmation time.

**Outputs Y:** They were classified into three categories:

- **Actual Output Ys:** The system's actual pill dispensing time (The time difference between the alarm is confirmed and the pills are detected by the PIR motion sensor).
- **Expected Output Ys:** The average system dispensing time is calculated based on the previously recorded values for pill fall detections. These values are previously stored in the context model, so they are quired and the average is calculated based on the average formula: Average confirmation time = **Σ Plausible Pill detection times / Number of Plausible Pill detection times**
- **Controlled Output Yp:** In this case, the system interacts with the user to see if the user extracted the pill, so in this case, the system could monitor that there was a failure in the PIR motion sensor. Otherwise, if the pill was not found by the user, maybe there was a failure in the solenoid push, or maybe the magazine was empty.

During the system being in the **1st round** (learning round), the **Output Discrepancy ΔY** is calculated and mapped to the **Dynamic Context C** presented at the time in which the output was not as expected as calculated.

In the pill dispensing use case, there are five external contextual elements C hypothesized to affect the alarm function:

- **External Temperature** might affect the performance of the PIR sensor. Since the temperature difference between the body and the ambient air is greater in colder climates, PIR sensors may be more sensitive. They might be more tolerant on a hot day. The fact that they are typically made for lights that only need to turn on at night when it is cooler means that this isn't a problem in most cases [26]. In parallel, the temperature might affect the solenoid valve. The maximum temperature of a solenoid valve can affect the service life of the valve. If the ambient temperature is too low, it will cause frost on the shell of the solenoid valve, which will cause problems such as sealing filler and making the valve unable to work normally [27].
- **External Humidity** might affect the performance of the solenoid valve. Electrical insulation will be impacted by relative humidity. Mold is simple to grow when the humidity is too high and the temperature is right, etc. Products with strong sealing abilities, such as waterproof, splash-proof, and dust-proof solenoid valves, should be chosen [27].

The other three internal context elements are the same as introduced in the motor positioning use case in section 3.6.3.1.

- **Internal Core Temperature**.
- **Internal Power consumption**
- **Internal Processor utilization**

After introducing the concept use cases with a different affecting contextual element. The Alarm use case was selected to be implemented as a prototype for this concept proof as it includes most of the parameters defined for this concept. Chapter 4 will introduce this prototype implementation.

# 4 Prototype Description

## 4.1 Prototype Implementation Diagram

In this Section, a prototype description for the concept was developed to map the context affecting the pill dispenser system to its dynamic behavior.

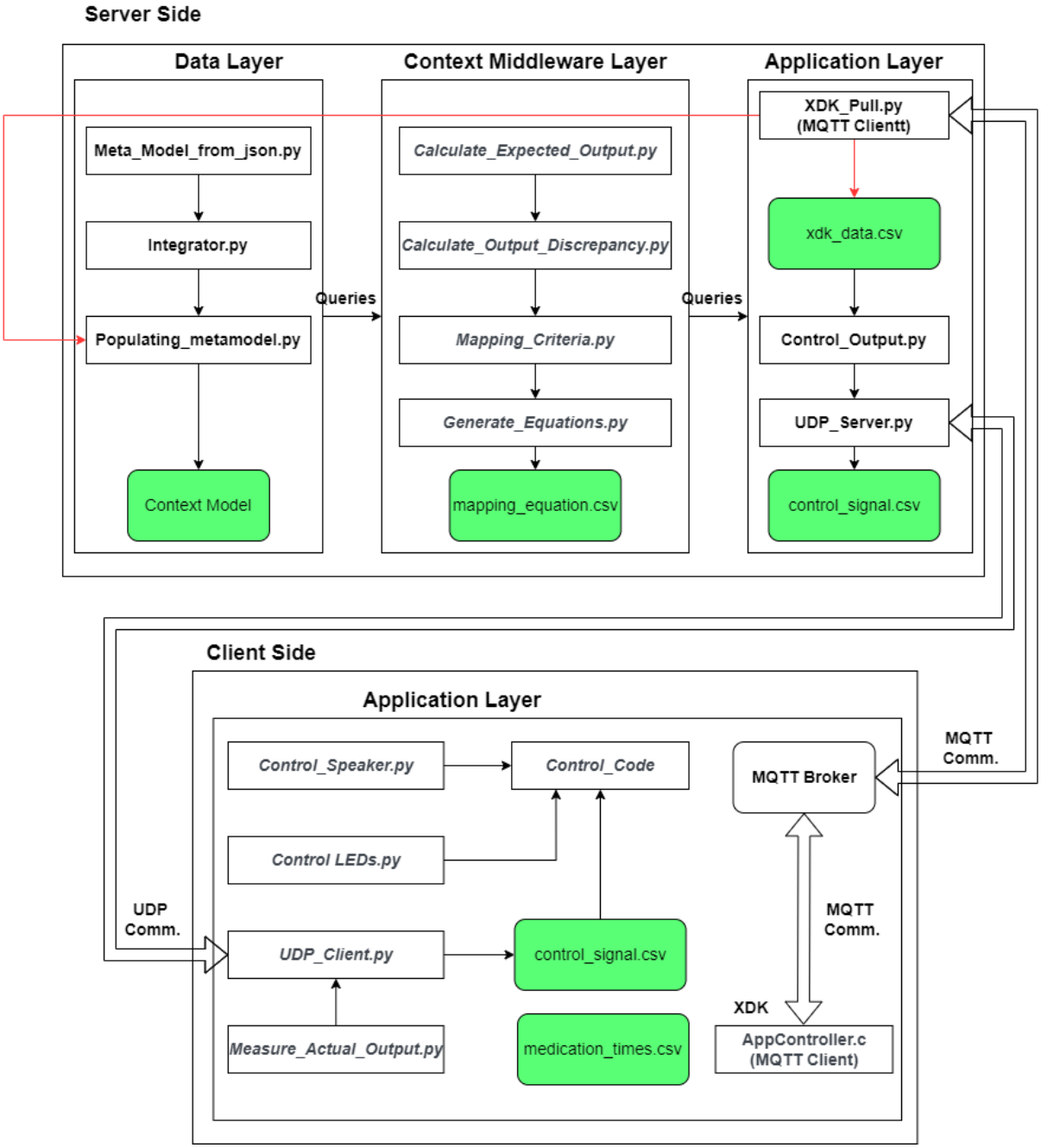


**Figure 21: Prototype Implementation Diagram**

Figure 21 shows the prototype implementation of the concept that has been implemented. The implementation is mainly divided into two sides with internal software components belonging to different software layers. **The server side**

represents the system graph representation and mapping process. It contains software components responsible for processing data and generating a control signal sent via a UDP communication to the hardware components under investigation in this work which are the alarm speaker and LEDs.

**The client side** represents the pill dispenser. It contains software components responsible for controlling the hardware components via the control signals coming from the server side. **On the server side**, there are three software layers. The data layer in which components are responsible for processing the data coming from different data sources and populating them in the current context model. Context middleware layer in which components are responsible for processing the data queried from the context data model and generating the mapping equation used furtherly to generate the hardware control signals. The application layer in which components are responsible for receiving the context values from the client side and generating the control signals, so they are sent back to the client side. **On the client side,** there is one software layer. The application layer in which the software components are responsible for measuring the hardware's actual output, and the context values measured by the XDK sensors and sending them to the server side to be processed either by creating-by-creating control signals or sending them back to the client side or by populating the context data to the context model, so they are stored in the system database for further processing. While receiving the control signals to control the hardware components which are the alarm speaker and LEDs in the implemented use case.

## 4.2 Server Side

As introduced in section 4.1, the server side has software components aiming at generating the hardware control signals. In this subsection, the server side’s software components will be introduced.

### 4.2.1 Data Layer

The software components for this software components already existed but were only updated by the current implementation to meet the objective of the current implementation. They are all python-based software components.

#### 4.2.1.1 Meta_Model_from_json

The software component is responsible for resetting and creating the system meta model which represents the graph structure for the system. It reads a .json file in which the meta model is structured as nodes that store node labels and properties, and relationships between graph nodes. It already existed, however, the current implementation added labels to all existing nodes to categorize them as either structure, behavior, internal context, or external context based on the FBS design framework introduced in the previous basic and conception concepts. It also added new nodes to represent the use cases introduced in the conception document and their

relationship. Motor positioning time and rotational speed are the behaviors for the motor structure, so they were connected by a HAS relationship. It also added new relationships between the nodes to represent their correlations. Figure 22 shows the Alarm node added to represent the alarm use case introduced in the conception document. In this case light, noise, user activity, and location were considered as the external context affecting the alarm function, so they were connected to the alarm node with AFFECTS relationship. A medication times node was added to represent the user's actual alarm confirmation time, so it was connected to the alarm node with an INPUT_TO relationship from which the system expected output is calculated. At the same type of medication times, nodes are considered as the system's actual output (user confirmation type), so the relationship type is considered "Bidirectional". The speaker and LEDs are the structures controlled but the alarm function, so they were connected from the alarm node by a CONTROLS relationship. Speaker sound level and LEDs brightness nodes were added to represent the behavior of these structures, so they were connected from their respective structures by a HAS relationship.

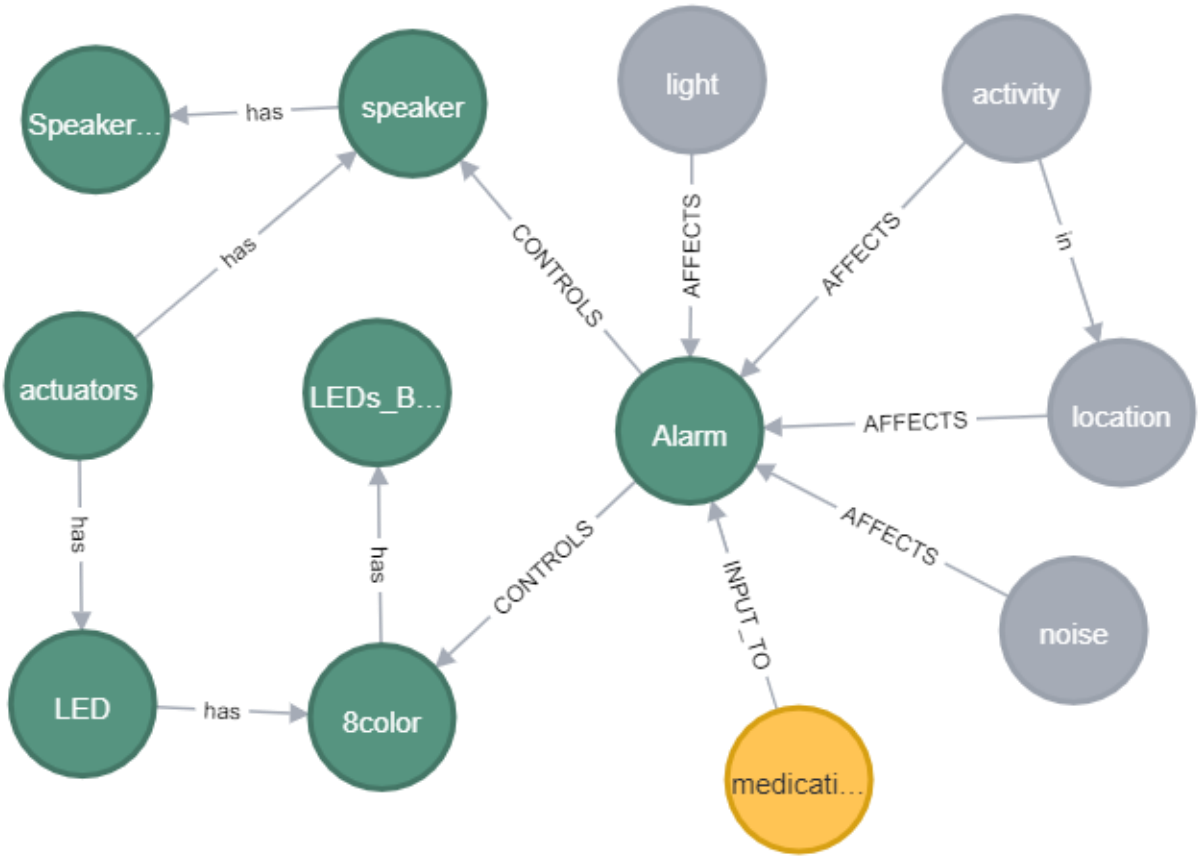


**Figure 22: Alarm Use case queried from context model**

In the same concept, Figure 23 shows the Position Motors node added to represent the motor positioning use case introduced in the conception document. Power and processor utilization was considered as the internal context affecting the motor positioning function, so they were connected to the Position Motors node by an AFFECTS relationship. Alarm times are considered as the function input, so it is connected to the Position Motors nodes by an INPUT_TO relationship. The motor

structure is controlled by the Position Motors function, so they are related by a CONTROLS relationship.

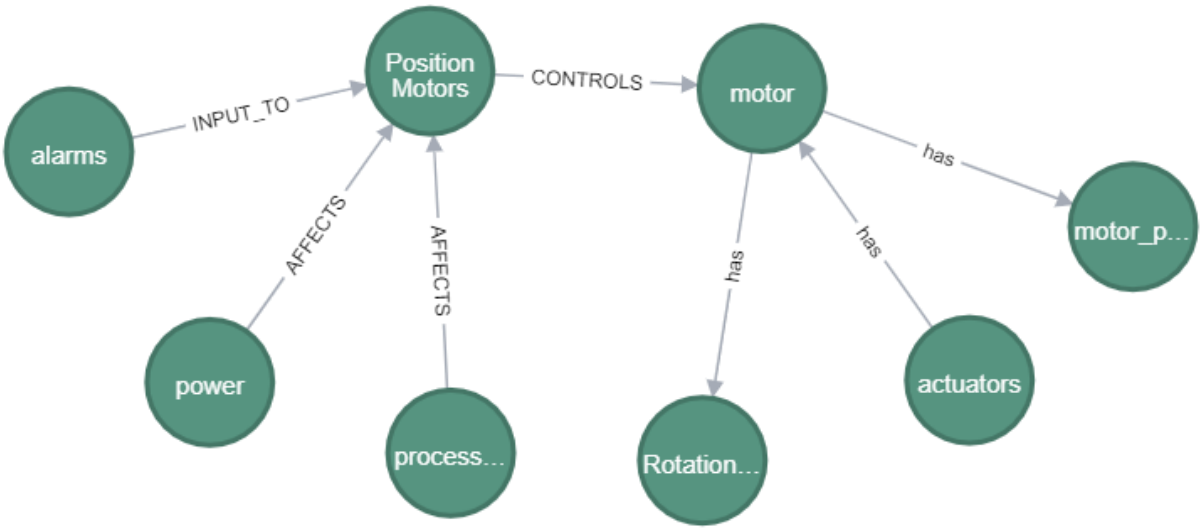


**Figure 23: Motor Positioning Use Case queried from context model**

Figure 24 shows Pill dispensing node added to represent the dispensing time use case introduced in the conception document. Power, core temperature, and processor utilization were considered as the internal context affecting the dispensing time function,

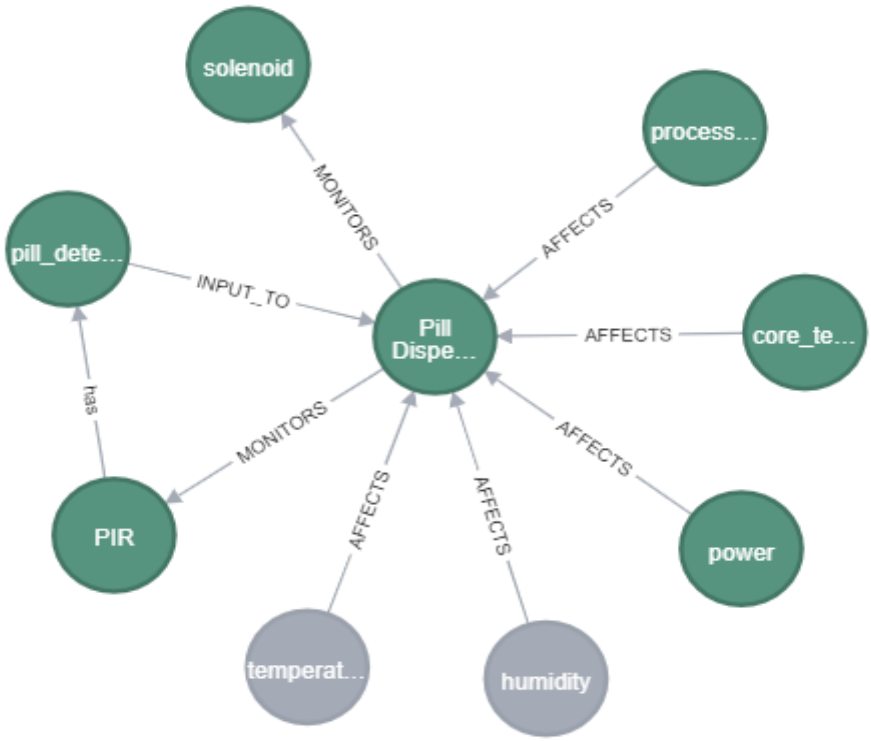


**Figure 24: Pill Dispensing Use Case queried from context model**

so, they were connected to the Pill dispensing node by an AFFECT relationship. Pill Detection Time is considered as the function input, so it is connected to the Pill dispensing nodes by an INPUT_TO relationship. The solenoid and PIR motion sensors are the structure monitored by the pill dispensing function, so they are connected with a MONITORS relationship.

#### 4.2.1.2 Integrator

This software component transforms various datasets into JSON datasets with structures that can be verified using the corresponding set schemas. It already existed, however, a method called *medication_times_to_json* was added by the current implementation to generate the JSON file for the medication_times.csv file in which the user's previous confirmation times are stored.

#### 4.2.1.3 Populating_metamodel

This software component is middleware that uses the created meta model and the created JSON datasets to populate the meta model with all of the values. It already existed, however, a method called *populate_model_medication_times* was added by the current implementation to populate the medication_times data to the current context model in which XDK data and user context data are stored.

### 4.2.2 Context Middleware

The software components for these software components implemented by the current implementation, to meet the objective of the introduced use cases. They are all python-based software components except for *Generate Equations* which was written via MATLAB environment.

#### 4.2.2.1 Calculate_Expected_Output

This software component is responsible for calculating the system's expected output. Currently, it has only one method called *calculate_average_ conformation_time* which does the following:

- Query the populated medication times stored in the context model.
- Calculate the user average confirmation times at different daytimes and output it as an expected confirmation time.

The below figure shows the results of the function in minutes with a threshold equal to 100 minutes, so any previous output with a confirmation time more than the threshold value is discarded.

```
Mornings: 0:25:17.942110 delay
Noons: 0:47:15.114129 delay
Evenings: 0:58:40.993997 delay
Nights: 0:28:58.803488 delay
Days: 0:36:49.929392 delay
```

These output values are returned in a dictionary data type, so they are furtherly considered as the expected output for the user confirmation time. In a future implementation, other methods can be added to calculate other functions' expected outputs from the system.

#### 4.2.2.2 Calculate_Output_Discrepancy

This software component is responsible for calculating the system output discrepancy. It has four methods as the following:

- *populate_actual_conformation_time_csv_data:* It Creates the .json files for the actual medication_times measured from the system and populates them to the context model.
- *calculate_discrepancy_in_conformation_time:* It Queries the user's actual medication_times stored in the graph at a certain timestamp and daytime and compares them with its relative daytime expected output, and returns the discrepancy value in a dictionary data type with discrepancy value, positive sign if the alarm was confirmed after the expected value, negative value if the alarm was confirmed before the expected value, and no sign if the alarm was not confirmed at all.
- write_discrepency_in_conformation_time_csv*:* It output the calculated output discrepancy at a certain timestamp and daytime into *confirmation_times_ discrepancy.csv* file.
- *write_discrepency_in_conformation_time_csv_graph:* It queries all the stored medication_times at different timestamps and medication times and writes all their respective discrepancies values into *confirmation_times_ discrepancy.csv* file.

The below figure shows the first 12 lines of output CSV from this software component.

| | |
|---|---|
| 1 | timestamp,daytime,discrepancy_in_user_confirmation_time |
| 2 | _23_07_2021T1300,noon,+1:51:03.967884 |
| 3 | _22_08_2021T0800,morning,-0:23:02.743904 |
| 4 | _21_08_2021T2100,night,+0:18:58.689868 |
| 5 | _21_08_2021T1700,evening,+1:43:23.548853 |
| 6 | _21_08_2021T1300,noon,+1:52:36.646762 |
| 7 | _21_08_2021T0800,morning,+0:10:55.023187 |
| 8 | _20_08_2021T2100,night,-0:17:58.278030 |
| 9 | _20_08_2021T1700,evening,-0:54:31.546763 |
| 10 | _20_08_2021T1300,noon,+0:21:45.264892 |
| 11 | _20_08_2021T0800,morning,-0:22:24.424471 |
| 12 | _19_08_2021T2100,night,not_opened |

#### 4.2.2.3 Mapping_Criteria

This software component is responsible for mapping the output discrepancies to their present context values. It has two methods as the following:

- *query_present_context_csv:* It queries the context values corresponding to the output of the discrepancies calculated by the previous software component and writes them on the *confirmation_times_discrepancy_with_context.csv* file.
- *mapping_equation_alarm_from_matlab:* It calculates a learned output discrepancy value based on the given context values and the mapping equation generated via MATLAB script introduced in subsection 4.2.2.4. It returns the output value in a dictionary data type with *discrepancy* value representing the learned discrepancy value, *noise_effect* representing the noise contribution in this learned output value, and *light_effect* representing the light contribution in this learned output value.

The below figures show the first 12 lines of output CSV from this software component.

| | |
|---|---|
| 1 | timestamp,daytime,discrepancy_in_user_confirmation_time,discrepancy_in_minutes,noise,light,activity,location |
| 2 | _23_07_2021T1300,noon,+1:51:03.967884,111.06613139999999,66.21335468,492.48,resting,indoors |
| 3 | _22_08_2021T0800,morning,-0:23:02.743904,-23.045731733333334,62.40969608,13.95,resting,indoors |
| 4 | _21_08_2021T2100,night,+0:18:58.689868,18.978164466666666,60.36483428,13.95,resting,indoors |
| 5 | _21_08_2021T1700,evening,+1:43:23.548853,103.39248088333333,60.56553204,13.95,resting,indoors |
| 6 | _21_08_2021T1300,noon,+1:52:36.646762,112.61077936666668,61.04268018,13.95,resting,indoors |
| 7 | _21_08_2021T0800,morning,+0:10:55.023187,10.917053116666667,65.66927246,13.95,resting,indoors |
| 8 | _20_08_2021T2100,night,-0:17:58.278030,-17.971300499999998,59.9655726,13.95,resting,indoors |
| 9 | _20_08_2021T1700,evening,-0:54:31.546763,-54.52577938333333,59.11544131,13.95,resting,indoors |
| 10 | _20_08_2021T1300,noon,+0:21:45.264892,21.754414866666664,65.4200109,13.95,resting,indoors |
| 11 | _20_08_2021T0800,morning,-0:22:24.424471,-22.40707451666667,62.55388139,13.95,resting,indoors |
| 12 | _19_08_2021T2100,night,not_opened,270.0,76.3508482,13.95,resting,indoors |

```
{'Discrepancy': 46.17118895143332,
'noise_effect': -0.11175574777852172,
'light_effect': -0.051301597566035126}
```

#### 4.2.2.4 Generate_Equations

This software component reads the output CSV file from the mapping criteria module introduced in section 4.2.2.3, normalizes these, and applies the coefficients estimation command to estimate the mapping equation. It writes output equation parameters (data mean value, standard deviation, coefficient values) in *the mapping_equation.csv* file and then plots the solution space. The result equation is $y= [B]_{1xn} \times [X]_{nx1}$ where the **X** row vector is the context vector, while **the B** column vector is the coefficients for the equation.

Based on the user data the following equation was generated:

$$y_{norm} = 0.1179\,x1\_norm + 0.0864\,x2\_norm$$

Figure 25 shows the mesh plot of the mapping equation where X and Y are the context values, while Z is the output learned discrepancy based on the previously recorded data and the estimated coefficients.

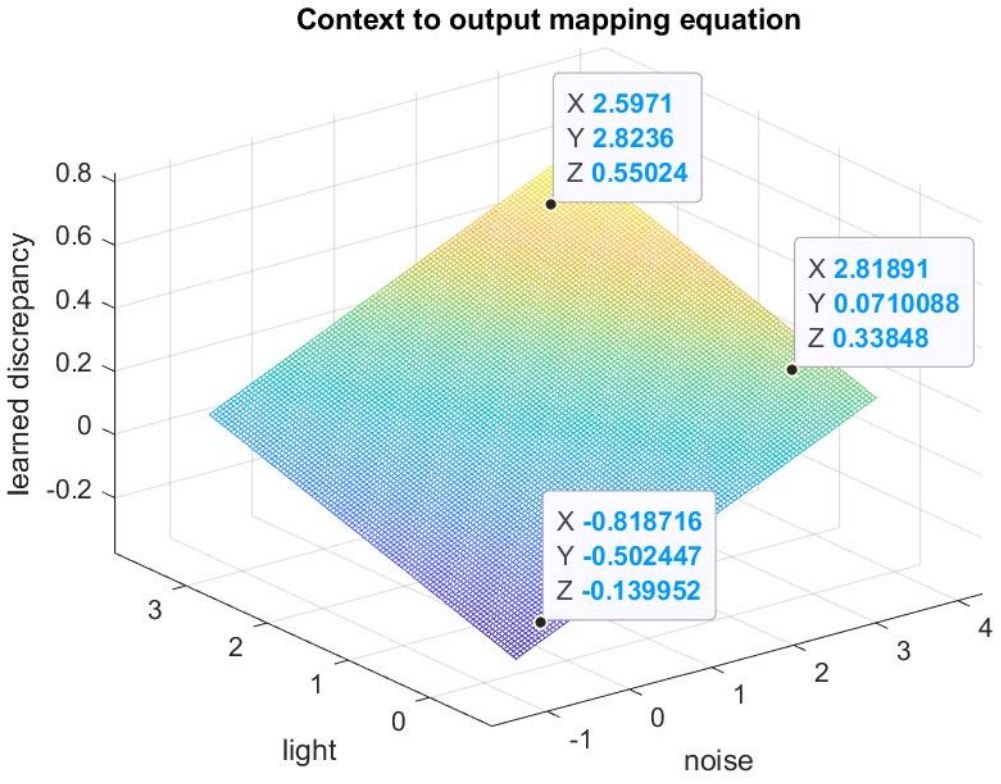


**Figure 25: Mapping Equation plot via MATLAB**

### 4.2.3 Server Application Layer

The software components for this software component are implemented by the current implementation, to meet the objective of the introduced use cases. They are all python-based software components.

#### 4.2.3.1 Control_Output

This software component is responsible for generating the control of the hardware structure based on the given context values and the learned discrepancy values calculated as the output of the estimated mapping equation introduced in 4.2.2.4. It has two methods as the following:

- *Control_PillDispenser_AlarmOutput:* It applies the mapping equation on the given context to calculate the discrepancy values. Then it calculates which contextual element has many contributions to the calculated learned discrepancy value, so the corresponding system structure has the higher part of the generated control signal. It categorizes the learned discrepancy into four levels of discrepancy (Too small, small, reasonable, and large) discrepancy. Based on the discrepancy level, the control signal gain is tuned. Then it queries the behavior nodes of the speaker and LEDs structures and sets the new speaker volume and LEDs brightness respectively.

- *Map_valueRatio_to_controlSignal:* It maps any input value from one range to another range. It is used to map the generated control signal from a percentage value to another value based on the controlled system structure.

The figure below shows the results of this software component when context vector = {"noise":91.078987, "light":8} was applied in dB, and lux respectively. This discrepancy value was categorized as large discrepancy based on it could be noted that the noise value is large compared to the light value. That was reflected in the noise and light ratio calculations and the generated control signal for the speaker [0,50] and LEDs [0,5].

```
Discrepancy: 100.71553703975894
Nosie Ratio: 88.55612946766668
Light Ratio: 11.443870532333317
Discrepancy level: Large Discrepancy
{'Speaker_control': 44.27806473383334,
'LEDs_control': 0.5721935266166658}
```

The figure below shows the results of this software component when context vector = {"noise": 30.790062, "light": 2000} was applied in dB, and lux respectively.

```
Discrepancy: 75.66837849637234
Nosie Ratio: 45.55995531877507
Light Ratio: 54.44004468122493
Discrepancy level: Reasonable Discrepancy
{'Speaker_control': 17.084983244540652,
'LEDs_control': 2.041501675545935}
```

The figure below shows the results of this software component when context vector = {"noise": 30.790062, "light": 500} was applied in dB, and lux respectively.

```
Discrepancy: 13.543482562200033
Nosie Ratio: 81.30961564846989
Light Ratio: 18.690384351530103
Discrepancy level: Small Discrepancy
{'Speaker_control': 20.32740391211747,
'LEDs_control': 0.46725960878825257}
```

The figure below shows the results of this software component when context vector = {"noise": 40, "light": 8} was applied in dB, and lux respectively.

```
Discrepancy: 9.596065063566833
Nosie Ratio: 89.57639121376322
Light Ratio: 10.42360878623678
Discrepancy level: Too small Discrepancy
{'Speaker_control': 11.197048901720402,
'LEDs_control': 0.13029510982795975}
```

#### 4.2.3.2 UDP_Server

This software component is responsible for handling the requests coming from the client side which in this case is the hardware control code of the pill dispenser. It was

designed in the form of UDP socket programming. The current implementation designed two types of commands or requests as the following:

- *Get_control_signals:* It requests the speaker and LEDs control signals from the server side. The server side pulls the context values sent by the BOSCH XDK sensor kit. Then it calculates the control signals and sent them back to the client side as a response to that request. The figure below shows the requested output:

```
UDP server up and listening
{'noise': 34.145271, 'light': 216}
Discrepancy: 7.766504397826708
Nosie Ratio: 95.05835475399367
Light Ratio: 4.941645246006327
Discrepancy level: Too small Discrepancy
```

- *Write_actual_output:* It requests a writing operation for the measured actual user confirmation time. These values are stored in a .csv file and then populated to the context model by *populate_actual_conformation_time_csv_data* as it was introduced in *4.2.2.2*. The figure below shows the requested output:

```
UDP server up and listening
['date', 'daytime', 'triggered_times', 'opened_times']
['2022-11-27', 'morning', '08:00', '2022-11-27 8:00:00']
Write request was handeled
```

#### 4.2.3.3 XDK_Pull

This software component is responsible for pulling the data coming from the XDK over an MQTT broker and storing them in xdk_data.csv to be mapped to a learned discrepancy value which is used for generating the hardware control signals as introduced in Control_Output.py in 4.2.3.1 and send to the client side over the UDP_Server introduced in 4.2.3.2. The following figure shows the output of this software component where the sensory data are pulled once every 5 seconds.

```
timestamp:   2022-11-29 16:48:00
xdk_id:2,Humidity:52,Pressure:96470,Temperature:29.071000,Light:11,Noise:24.354382,
timestamp:   2022-11-29 16:48:05
xdk_id:2,Humidity:52,Pressure:96473,Temperature:29.071000,Light:11,Noise:24.464334,
timestamp:   2022-11-29 16:48:11
xdk_id:2,Humidity:52,Pressure:96472,Temperature:29.061000,Light:11,Noise:24.641621,
timestamp:   2022-11-29 16:48:16
xdk_id:2,Humidity:52,Pressure:96471,Temperature:29.071000,Light:11,Noise:24.066664,
timestamp:   2022-11-29 16:48:21
xdk_id:2,Humidity:52,Pressure:96472,Temperature:29.061000,Light:11,Noise:27.669260,
timestamp:   2022-11-29 16:48:26
```

## 4.3 Client Side

As introduced in section 4.1, the client side has software components aiming at controlling the hardware functions and measuring the system's actual output. In this subsection, the client side's software components will be introduced.

### 4.3.1 Client Application Layer

The software components for this software component are implemented by the current implementation, to meet the objective of the introduced use cases. They are all python-based software components except for Appcontroller.c was written in C programming as a part of a C project programmed and flashed on BOSCH XDK using the XDK workbench tool

#### 4.3.1.1 Measure_Expected_Output

This software component is responsible for measuring the system's actual output. It was coded as a python class so it can be easily integrated with the existing hardware control code. It has six methods as the following:

- *__init__*: It initializes the class variables (alarm triggered and confirmed time).
- *get_user_confirmation_time:* It calculates the difference between the alarm triggered and confirmed alarm.
- *set_alarm_triggered:* It sets the value of the alarm triggered time.
- *get_alarm_triggered:* It gets the value of the alarm triggered time.
- *set_alarm_confirmed:* It sets the value of the alarm confirmed time.
- *write_medication_times_csv:* It writes the measured actual output in a medication_times_actual.csv.

#### 4.3.1.2 Control_Speaker

This software component is responsible for controlling the speaker volume output for the alarm use case based on the present context value. It was coded as a python class so it can be easily integrated with the existing hardware control code. It has three methods as the following:

- *__init__:* It initializes the class variables (audio Mixer, and default volume value).
- *volumeMasterControl:* It sets the speaker output volume based on the speaker control signal calculated by the server application layer.
- *extract_control_signal:* It reads the 'control_signals_daytime.csv in which the hardware control signals are stored and extract the speaker control signal.

#### 4.3.1.3 Control_LEDs

This software component is responsible for controlling the LEDs' brightness value by changing the duty cycle of the PWM signal applied to the LEDs.

It already existed in the hardware control code; however, it was updated with the three methods as the following:

- *turn_on_blue:* It sets the LEDs to blue color (LEDs GPIO pins).
- *turn_off_blue:* It resets the LEDs' blue color.
- *extract_control_signal:* It reads the 'control_signals_daytime.csv in which the hardware control signals are stored and extract the LEDs control signal.

#### 4.3.1.4 Control_Code

This software component is responsible for controlling all hardware components. It already existed in the hardware control code; however, it was updated by adding the output measurement, speaker control, and LEDs control features. The following three methods were updated:

- *position_actor:* After the actuators are positioned in front of the magazines according to the current alarm day and daytime, the alarm triggered time is set by the value of the current time. Then a *Get_control_signals* request to the server side and the control signals for the speaker and LEDs are received and extracted.
- *open_magazin:* Once the alarm is confirmed and the magazine is opened, the alarm confirmed time is by the value of the current time.
- *wait_for_confirmation:* In addition to blinking LEDs with a frequency of 0.5 Hz, a PWM concept was coded to manipulate the LEDs' brightness via manipulating the PWM duty cycle based on the LEDs' control signal.

#### 4.3.1.5 UDP_Client

This software component is responsible for sending client requests introduced in UDP_Server 4.2.3.2 and processing the returned value. It was coded as a python class so it can be easily integrated with the existing hardware control code. It has three methods as the following:

- *__init__*: initializes the class variables (IP address, port, buffer size, and socket).
- *UDP_Send_Client:* Send a UDP datagram as a pair of (Command, Value) to the Server side.
- *UDP_Recv_Client:* Receive a UDP datagram as a pair of (Message, Content) from the server side.

#### 4.3.1.6 APPController

This software component is part of the SendDataOverMQTT project flashed on the BOSCH XDK. It is responsible for reading the sensory data from the XDK. It has an MQTT client which sends these sensory data to the server side over an MQTT broker.

The following figure shows the output console of the XDK workbench tool:

```
INFO | XDK DEVICE 1: Started collecting WLAN Rx Statistic
INFO | XDK DEVICE 1: IP address of device  192.168.178.34
INFO | XDK DEVICE 1:               Mask    255.255.255.0
INFO | XDK DEVICE 1:               Gateway 192.168.178.1
INFO | XDK DEVICE 1:               DNS     192.168.178.1
INFO | XDK DEVICE 1: Connected to WPA network successfully
INFO | XDK DEVICE 1: MqttEventHandler : Event - 0
INFO | XDK DEVICE 1: ServalMqttAgent:SubscribeToTopic - Subscribing to topic: mqtt/environment_parameters, Qos: 1
INFO | XDK DEVICE 1: MqttEventHandler : Event - 8
INFO | XDK DEVICE 1:  38
INFO | XDK DEVICE 1:  32.048405
INFO | XDK DEVICE 1:  92160
INFO | XDK DEVICE 1:  96358
INFO | XDK DEVICE 1:  217
INFO | XDK DEVICE 1:  *******Light sensor application callback received*******
INFO | XDK DEVICE 1: MqttEventHandler : Event - 14
INFO | XDK DEVICE 1: AppMQTTSubscribeCB : #1, Incoming Message:
INFO | XDK DEVICE 1:   Topic: mqtt/environment_parameters
INFO | XDK DEVICE 1:   Payload:
INFO | XDK DEVICE 1: """
INFO | XDK DEVICE 1: xdk_id:2,Humidity:38,Pressure:96358,Temperature:21.791000,Light:92,Noise:32.048405,
INFO | XDK DEVICE 1: """
```

# 5 Integration and Test

After the prototype implementation was developed, it was tested via the results of the python IDE console. There was a step of testing it on the pill dispenser hardware that exists in IAS. The following steps were followed to integrate the prototype into the system's existing hardware control codes:

- The software components of the server side were run on the laptop.
- The software components of the client side were copied on the pill dispenser raspberry pi and then run through the existing pill dispenser application.
- Both sides were connected to the same internet network, so they can communicate via both UDP and MQTT communication protocols.
- The Steps of running the setup are documented in detail in the user manual document attached with the thesis.

For the testing of the prototype implementation different context conditions were applied on the system as the following in this table where **X =** [noise, light] is the context vector applied in different cases and measured via BOSCH XDK:

**Table 2: Output of Mapping external dynamic Context to Adaptive Alarm**

| Case | Condition | Context vector X [dB, lux] | Speaker Output % | LEDs Brightness % |
|---|---|---|---|---|
| 1 | [low noise, low light] | [35,240] | 66% | 60% |
| 2 | [high noise, low light] | [70,240] | 74% | 60% |
| 3 | [normal noise, normal light] | [50,1500] | 66% | 80% |
| 4 | [low noise, high light] | [35,36700] | 50% | 100% |

These results can be verified by the demo videos attached with the thesis taking into consideration that they were captured at night when the IAS lab ceil lights were turned on. In **case 1**, low noise [30-40] dB and low light value [230-250] lux conditions were applied. As the noise and light context here have a similar contribution to the system the speaker and LEDs output were adapted also to similar output levels which were 66% for the speaker and 60% for the LEDs. In **case 2,** the noise context was increased to a high value [60-80] dB, while the light context remains the same, so the speaker output was adapted to a higher value of 74%. In **case 3,** the noise context was decreased to the normal value [40-60] dB, while the light context was increased to the normal value [1400-1600] lux with more contribution compared to the noise context, so the speaker output was again adapted to 66% while the LEDs. In **case 4,** the noise context was much decreased to a low value [30-40] dB, while the light context was

much increased to a very high value of 36700 lux, so the noise context has a very small contribution to the output discrepancy compared to the light context contribution. In this case, the speaker sound level was adapted to its minimum value of 50%, while the LEDs' brightness was adapted to its maximum brightness value of 100%.

By investigating these different context cases we can conclude that the concept was proofed, and the prototype implementation was accepted and integrated into the existing system implementation.

# 6 Conclusion and Future Work

In this research project, a concept of mapping the external system's concept to its dynamic behavior at run time was developed for the pill dispenser as medication assistance for elderly people. First, the system's current context graph was updated based on the FBS framework design by Gero, so the context model meets the project objective of mapping the system's external context to its dynamic behavior at runtime. The graph model elements were categorized into either structure, function, structure, internal context, or external context via adding labels. New nodes, and relationships, were added to the context model to represent the system functions and their relationships with the system structures and system behaviors. Based on these graph updates, a query procedure was developed to access the data stored in the graph in a general and unique manner for system functions.

After context model adjustment, three use cases were conceptually developed. They were alarm, motor positioning, and pill dispensing use cases. Following the introduced concept, each system function has an actual output to be measured, and an expected output to be calculated based on the previously recorded data stored on the context model.

In the system 1$^{st}$ round, the difference between these two outputs is calculated as an output discrepancy. Based on the previous recorded output discrepancies and context values a mathematical equation was estimated to map the output discrepancy to the concurrent external context values.

In the system 2$^{nd}$ round (at runtime), the present system context value is queried and passed to the estimated mapping equation, so a learned discrepancy value is calculated and passed to a control rule to generate a control signal applied on the system structure controlled by a specific system function, so the system expected, and actual output comes close. The alarm use case was implemented as a proof of concept, and the alarm speaker and LEDs structure were adapted to the external environment noise and light respectively.

The implemented software modules were classified into server side and client side. Both sides communicate via UDP communication protocol. Then, they were integrated with the existing system control codes, so the alarm function adaptability is achieved.

For future recommendations, the dataset on which the mapping equation was estimated should be enlarged, so that the mapping between the context values and the calculated discrepancy value is more accurate. The UDP communication between the server side and the client side is blocked. It can be furtherly modified to be non-blocking, so it does not stop the pill dispenser code from performing its normal operations. The motor positioning and pill dispensing use case can be also implemented to give the system more adaptability to different internal and external affecting contextual elements.

# 7 Bibliography


| | |
|---|---|
| [1] | *RMIT University. (July 2014)* Retrieved June 2022 from What are cyber-physical systems? - RMIT University |
| [2] | Jazdi, N. (2014, May). Cyber-physical systems in the context of Industry 4.0. In *2014 IEEE international conference on automation, quality and testing, robotics* (pp. 1-4). IEEE. |
| [3] | Sahlab, N., Jazdi, N., & Weyrich, M. (2021). An Approach for Context-Aware Cyber-Physical Automation Systems. *IFAC-PapersOnLine*, *54*(4), 171-176. |
| [4] | Dey, A. K. (2001). Understanding and using context. *Personal and ubiquitous computing*, *5*(1), 4-7. |
| [5] | Sahlab, N., Jazdi, N., & Weyrich, M. (2020, September). Dynamic Context Modeling for Cyber-Physical Systems Applied to a Pill Dispenser. In *2020 25th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)* (Vol. 1, pp. 1435-1438). IEEE. |
| [6] | Sahlab, N., Jazdi, N., & Weyrich, M. (2020, September). Dynamic Context Modeling for Cyber-Physical Systems Applied to a Pill Dispenser. In *2020 25th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)* (Vol. 1, pp. 1435-1438). IEEE. |
| [7] | P. Rosenberger and D. Gerhard, “Context-awareness in industrial applications: definition, classification and use case,” Procedia CIRP, vol. 72, pp. 1172–1177, 2018, doi: 10.1016/j.procir.2018.03.242 |

| | |
|---|---|
| [8] | Sahlab, N., Jazdi, N., & Weyrich, M. (2022). An Overview on Designs and Applications of Context-Aware Automation Systems. *arXiv preprint arXiv:2207.02995*. |
| [9] | M. Knappmeyer, S. Kiani, E. Reetz, N. Baker, and R. Tönjes, “Survey of Context Provisioning Middleware,” IEEE Communication Surveys and Tutorials, vol. 15, 2012, doi: 10.1109/SURV.2013.010413.00207. |
| [10] | X. Li, “Context Aware Middleware Architectures: Survey and Challenges,” Sensors, vol. 15, 2015, doi: 10.3390/s150820570. |
| [11] | D. Brickley and R. Guha, "Rdf vocabulary description language 1.0: Rdf schema", *W3C W3C Recommendation*, Feb 2004, [online] Available: http://www.w3.org/TR/rdf-schema. |
| [12] | S. Bechhofer, F. van Harmelen, J. Hendler, I. Horrocks, D. L. McGuinness, P. F. Patel-Schneider, et al., "OWL web ontology language reference", *World Wide Web Consortium (W3C) W3C Recommendation*, February 2004, [online] Available: http://www.w3.org/TRlowl-ref/. |
| [13] | Gero, J. S., & Kannengiesser, U. (2004). The situated function–behavior–structure framework. *Design studies*, *25*(4), 373-391. |
| [14] | Wang, Y., Cen, Y., & Shen, X. STUDY ON CONSTRUCTION OF FBS-MODEL BASED PRODUCT DESIGN ONTOLOGY. |
| [15] | *Semantic Technologies*. (n.d.). International Center for Computational Logic. Retrieved December 24, 2022, from https:////iccl.inf.tu-dresden.de/web/Semantische_Technologien/en Semantic Technologies – International Center for Computational Logic (tu-dresden.de) |

| | |
|---|---|
| [16] | *Knowledge Graphs - Neo4j*. (2021, August 25). Retrieved October 23, 2022, from Knowledge Graphs (neo4j.com) |
| [17] | Neo4j Developer. (2018, August 24). *Graph Modeling Guidelines - Developer Guides*. Neo4j Graph Data Platform; neo4j.com. https://neo4j.com/developer/guide-data-modeling/ |
| [18] | Neo4j Developer. (2018, November 8). *Modeling Designs - Developer Guides*. Neo4j Graph Data Platform; neo4j.com. https://neo4j.com/developer/modeling-designs/ |
| [19] | Neo4j Developer. (2018, October 26). *Graph Modeling Tips - Developer Guides*. Neo4j Graph Data Platform; neo4j.com. https://neo4j.com/developer/modeling-tips/ |
| [20] | Neo4j Developer. (2018, December). *Getting Started with Cypher - Developer Guides*. Neo4j Graph Data Platform; neo4j.com. https://neo4j.com/developer/cypher/intro-cypher/ |
| [21] | Sahlab,N., Weyrich, M (2022). User Requirements Specification. |
| [22] | Sahlab, N., Jazdi, N., & Weyrich, M. (2020). An intelligent medication assistance system. *Proceedings on Automation in Medical Engineering*, *1*(1), 031-031. |

| | |
|---|---|
| [23] | Sahlab, N., Jazdi, N., Weyrich, M., Schmid, P., Reichelt, F., Maier, T., ... & Kalka, G. (2019, September). Development of an intelligent pill dispenser based on an IoT approach. In *International Conference on Human Systems Engineering and Design: Future Trends and Applications* (pp. 33-39). Springer, Cham. |
| [24] | *System monitoring | Dynatrace*. (2020, October 12). Dynatrace. Retrieved October 23, 2022, from System monitoring \| Dynatrace |
| [25] | *Copperhilltech.* (2020). The Operating Temperature for A Raspberry Pi. Retrieved October 23, 2022, from The Operating Temperature For A Raspberry Pi – Technologist Tips.pdf (copperhilltech.com) |
| [26] | *Lighting Info.* (30 June 2022). Does temperature affect the motion sensor? Retrieved November 05, 2022, from Does Temperature Affect Motion Sensors? - LED & Lighting Info (ledlightinginfo.com) |
| [27] | *YUYAO NO.4 INSTRUMENT FACTORY. (28 September 2020). Requirements for temperature and humidity of solenoid valve during operation.* Retrieved November 05, 2022, from Requirements for temperature and humidity of solenoid valve during operation - Knowledge - Yuyao No.4 Instrument Factory (chinasolenoidvalve.com) |

# Declaration of Compliance

I hereby declare to have written this work independently and to have respected in its preparation the relevant provisions, in particular those corresponding to the copyright protection of external materials. Whenever external materials (such as images, drawings, text, and passages) are used in this work, I declare that these materials are referenced accordingly (e.g., quote, source) and, whenever necessary, consent from the author to use such materials in my work has been obtained.

Signature:

Stuttgart, on the 30.12.2022